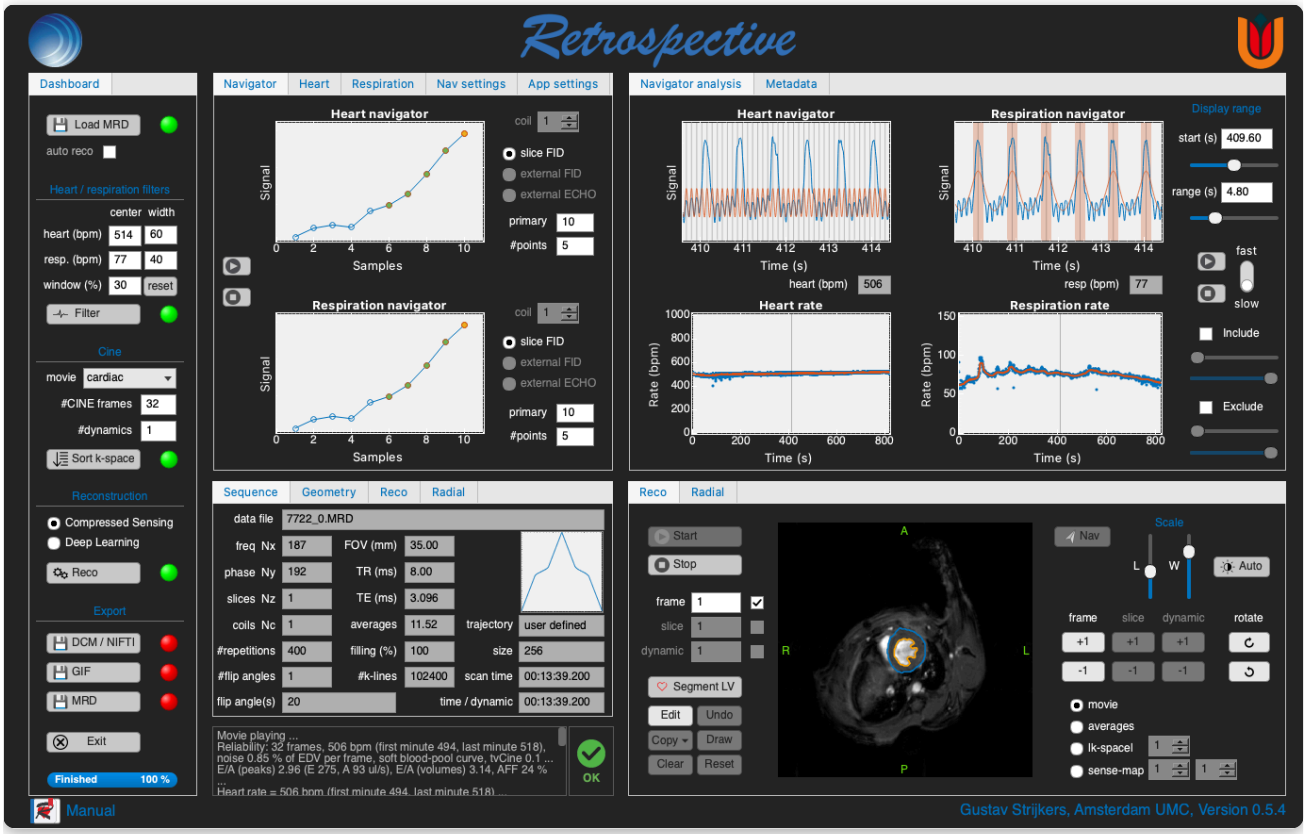



## Table of contents

1. Overview
2. Contact
3. Disclaimer
4. Citing the use of Retrospective
5. Installation
6. Running the software
7. Basic operation
8. Automatic reconstruction
9. Sounds and colors
10. Credits
11. Appendix: how E/A is calculated

## 1. Overview

Retrospective is an application with a graphical user interface to reconstruct self-gated cardiac or respiratory CINE MRI acquired with an MR Solutions preclinical MRI system. Reconstruction is based on compressed sensing, or on a trained deep learning network, after retrospective sorting of the data into cardiac and/or respiratory time frames, using a navigator signal for cardiac and respiratory synchronization. The left ventricle can be segmented automatically in the reconstructed short-axis CINE images.

Retrospective is a stand-alone application: everything it uses is inside it, and nothing else has to be installed.

## 2. Contact

Retrospective and this manual are written by:

Gustav Strijkers
Amsterdam UMC
Biomedical Engineering and Physics
Amsterdam, the Netherlands
g.j.strijkers@amsterdamumc.nl

## 3. Disclaimer

This is a beta version. The software has been extensively tested using mouse data acquired with an MR Solutions preclinical 7.0T/24cm system. However, this does not warrant that the functions contained in the app will meet your requirements or that the operation of the app will be uninterrupted or error-free. In case of questions or issues, please contact Gustav Strijkers.

The Gibbs ringing suppression (after Kellner et al.) and the radial density compensation (after Zwart et al.) are included for research use.

## 4. Citing the use of Retrospective

Retrospective is free of use. However, proper referencing of the use of Retrospective in scientific publications is highly appreciated. Citations to Retrospective can be done using the following reference (link):

Daal, M.R.R., Strijkers, G.J., Calcagno, C., Garipov, R.R., Wüst, R.C.I., Hautemann, D., Coolen, B.F. Quantification of Mouse Heart Left Ventricular Function, Myocardial Strain, and Hemodynamic Forces by Cardiovascular Magnetic Resonance Imaging. J. Vis. Exp. (171), e62595, doi:10.3791/62595 (2021).

## 5. Installation

### macOS

Requirements: a Mac with Apple silicon (M1 or later), macOS 26 or later.

Retrospective comes as a disk image, `Retrospective-0.6.1-macOS-arm64.dmg`. Nothing else has to be installed: Python and everything the app uses are inside it.

1. Open the disk image and drag **Retrospective** onto the Applications folder in its window.
2. Open it from Applications. The app is not signed with an Apple Developer ID, so the first time macOS refuses it ("Apple could not verify ..."); click **Done**. Then open **System Settings > Privacy & Security**, scroll down to the line saying Retrospective was blocked, click **Open Anyway** and confirm. From then on it opens as any other app.

   Instead of step 2, the same in Terminal, once:

   ```
   xattr -dr com.apple.quarantine /Applications/Retrospective.app
   ```

The first start can take a little longer: macOS checks the app once.

### Windows

Requirements: a PC with Windows 10 or 11, 64-bit.

Retrospective comes as an installer, `Retrospective-0.6.1-setup.exe`. Nothing else has to be installed: Python and everything the app uses are inside it.

1. Run the installer. It is not signed, so the first time Windows may stop it ("Windows protected your PC"): click **More info** and then **Run anyway**.
2. The installer puts the app in `%LOCALAPPDATA%\Programs\Retrospective` for the current user, without administrator rights (installing it for all users is an option), with a Start-menu entry and, if you choose, a desktop shortcut. At the end it can start the app.

To remove the app, use **Uninstall Retrospective** in the Start menu, or **Settings > Apps**.

### The deep learning reconstruction: the first start (macOS)

The deep learning reconstruction runs fastest on the Apple GPU, with PyTorch, in a Python environment of the app's own: the folder `venv_retropy` in your home folder. When it is not there, the app asks at its start, before its window opens, whether to set it up (**Create python environment**: **Create** / **Cancel**). On **Create** a window shows how far the setup has gotten. It is a few hundred megabytes to download and takes a few minutes; it happens once.

You do not need a Python of your own: when the computer has none the app can use, it installs one for itself in the folder `.retropy-python` in your home folder. None of this needs administrator rights, and nothing else on the computer is touched. A log of the setup is kept in `retropyPythonSetup.log` in the temporary folder.

Cancel the setup, or answer **Cancel**, and the deep learning reconstruction runs inside the app on the processor instead, which takes longer (about 3–4 times). The app asks again at its next start.

On Windows nothing has to be set up: the deep learning reconstruction runs inside the app, on the graphics card when it can (any DirectX 12 card), else on the processor.

### The LV segmentation

The LV segmentation runs inside the app on every computer: nothing to install. It uses the graphics card when there is one, else the processor.

### BART (optional)

With BART, the Berkeley Advanced Reconstruction Toolbox (https://mrirecon.github.io/bart/), the compressed sensing reconstruction runs with BART; without it, with the app's own engine, based on sigpy. Both give practically the same images; BART is faster, especially with GPU support. One combination of settings runs with sigpy even with BART: the data-derived temporal basis with TVcine or TVdyn (Step 6, Reconstruction), which only sigpy can do.

At its start the app says in its message box which one it uses: "BART toolbox detected ..." or "WARNING: BART toolbox not available, the sigpy engine is used ...", and whether BART was compiled with GPU support. A BART that does not answer within 30 seconds at the start counts as not available (or as without GPU support), and the message box says so. Retrospective is tested with BART 1.0.00.

#### Installing BART on macOS with MacPorts

1. Install the Xcode command line tools:

   ```
   xcode-select --install
   ```

2. Install MacPorts from https://www.macports.org/.
3. Install the packages:

   ```
   sudo port install fftw-3-single gcc14 libpng openblas flock gmake coreutils l
   ibblocksruntime
   ```

4. Download and unpack BART:

   ```
   wget https://github.com/mrirecon/bart/archive/v1.0.00.tar.gz
   tar xvfz v1.0.00.tar.gz
   cd bart-1.0.00
   ```

5. Build and install:

   ```
   export PATH=/opt/local/libexec/gnubin:$PATH
   CC=gcc-mp-14 gmake
   sudo gmake install
   ```

   The compiler name must match the gcc version installed in step 3: gcc14 gives gcc-mp-14, gcc15 gives gcc-mp-15.

6. Check that it works:

   ```
   bart version
   ```

#### Installing BART on macOS with Homebrew

1. Install the Xcode command line tools: `xcode-select --install`

2. Install Homebrew:

   ```
   /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
   ```

3. Install the packages BART needs:

   ```
   brew install gcc libpng fftw openblas make llvm libomp
   ```

   The Homebrew package that provides GNU make is called `make`; it installs the program under the name `gmake`.

4. Download and unpack BART as above (step 4 of MacPorts).
5. Build and install:

   ```
   CC=gcc-15 MACPORTS=0 gmake
   sudo gmake install
   ```

   The compiler name must match the version Homebrew installed; check it with `ls /opt/homebrew/bin/gcc-*`.

6. Check that it works: `bart version`

#### How Retrospective finds BART on macOS

In the normal case there is nothing to configure: `sudo make install` puts BART in `/usr/local/bin`, which is where the app looks first. It looks in these folders, in this order:

| Folder | |
|---|---|
| `/usr/local/bin` | where BART's own `make install` puts it |
| `/usr/bin` | |
| `/opt/homebrew/bin` | Homebrew's |
| `/opt/local/bin` | MacPorts' |

An app opened from the Finder or the Dock does not see the settings of your shell (such as `TOOLBOX_PATH` or `PATH` in your shell profile). If your `bart` is in another folder, a link to it in `/usr/local/bin` lets the app find it. In Terminal, once, with the folder your bart is in instead of `/path/to/bart` (Terminal asks for your password):

```
sudo mkdir -p /usr/local/bin
sudo ln -s /path/to/bart/bart /usr/local/bin/bart
```

#### Installing BART on Windows 11

On Windows, BART runs inside the Windows Subsystem for Linux (WSL); the app calls it as `wsl bart`.

1. Install WSL. Start a Windows PowerShell as administrator and run:

   ```
   wsl --install
   ```

   This enables the feature, installs WSL 2 and installs Ubuntu in one step. A restart is needed. The first time you start the Linux prompt you are asked to create a user account. On older systems, or if `wsl --install` is not recognized, see https://pureinfotech.com/install-wsl-windows-11/.

2. Update the Linux distribution (everything from here on is typed at the Linux prompt, not in PowerShell):

```
sudo apt-get update
sudo apt-get dist-upgrade
```

3. Install the packages BART needs (gcc 12 or newer):

```
sudo apt-get install make gcc libfftw3-dev liblapacke-dev libpng-dev libopenb
las-dev gfortran
```

4. Download and unpack BART:

```
wget https://github.com/mrirecon/bart/archive/v1.0.00.tar.gz
tar xvfz v1.0.00.tar.gz
cd bart-1.0.00
```

5. Build and install (for GPU support, follow the next section instead):

```
make
sudo make install
```

6. Check that it works, at the Linux prompt `bart version`, and from a Windows command prompt `wsl bart version`. Both should print a version number.

**Building BART with GPU support on Windows (provisional, highly recommended)**

The reconstruction is considerably faster on a GPU.

1. Install the NVIDIA driver in Windows, following https://docs.nvidia.com/cuda/wsl-user-guide/index.html. The Windows driver is what provides GPU access inside WSL: do not install a Linux display driver inside WSL.
2. Install the CUDA toolkit inside WSL. Get the current installer from https://developer.nvidia.com/cuda-downloads, choosing Linux → x86_64 → WSL-Ubuntu. For CUDA 12.8:

```
wget https://developer.download.nvidia.com/compute/cuda/12.8.0/local_installe
rs/cuda_12.8.0_570.86.10_linux.run
sudo sh cuda_12.8.0_570.86.10_linux.run
```

3. Find the compute capability of your card at https://developer.nvidia.com/cuda-gpus. BART has to be compiled for the GPU generation it will run on; a binary built for the wrong generation compiles without complaint and then fails at the first calculation.

| GPU generation | Example cards | Value to use |
|---|---|---|
| Pascal | GTX 1060, 1080, Titan X | `sm_61` |
| Turing | RTX 2060 – 2080 | `sm_75` |
| Ampere | RTX 3060 – 3090, A100 | `sm_80` / `sm_86` |
| Ada Lovelace | RTX 4060 – 4090 | `sm_89` |
| Blackwell | RTX 5070 – 5090 | `sm_120` |

4. Set the environment variables, in the shell you will build in (adjust `CUDA_BASE` to your CUDA version and `GPUARCH_FLAGS` to the value of step 3):

```
export CUDA=1
export CUDA_BASE=/usr/local/cuda-12.8
export CUDA_LIB=lib64
export GPUARCH_FLAGS="-arch=sm_61"
export PATH=$CUDA_BASE/bin:$PATH
export LD_LIBRARY_PATH=$CUDA_BASE/$CUDA_LIB:$LD_LIBRARY_PATH
```

5. Build:

```
make -j$(nproc) CUDA=1 CUDA_BASE=$CUDA_BASE CUDA_LIB=$CUDA_LIB GPUARCH_FLAGS
="$GPUARCH_FLAGS"
sudo make install
```

   If the build fails at the linking stage with an error about libstdc++ or libgcc, point the linker at the installed gcc (check the version with `ls /usr/lib/gcc/x86_64-linux-gnu/`) and build again:

```
export LDFLAGS="-L/usr/lib/gcc/x86_64-linux-gnu/13 $LDFLAGS"
```

6. Test the GPU build: `make utest_gpu`

What matters for Retrospective is that `bart` is on the PATH inside your Linux distribution; `sudo make install` takes care of that.

## 6. Running the software

Open **Retrospective** from the Applications folder (macOS), or from the Start menu or the desktop shortcut (Windows). The Retrospective logo appears while the app loads, and the window follows within seconds; the app then checks for BART and for the deep learning reconstruction, and "Ready" in the message box says it is done.

A scan given on the app's command line (an MRD file or a scan folder), as a program that starts the app with a scan does, is loaded at once, as **Load MRD** would load it. When the app is open already, a scan given this way goes to the open window instead: the window comes to the front and loads it, or loads it when the step under way ends.

**Newer versions.** At its start the app asks GitHub (https://github.com/Moby1971/MRI-apps) whether there is a newer release of it. The message box then shows the current and the newest version and, when there is a newer one, where to download it. Without a network connection it says that the version could not be checked, and the app works as usual. New versions also arrive by themselves while **auto update** in the App settings tab is ticked (see "The App settings tab"). With **auto update** unticked, a window at the start offers a newer version, unless **update notice** is unticked. `checkForUpdates` in the settings file switches the check off; `versionCheckTimeout` sets how many seconds it waits for GitHub (5).

**This manual.** The **Manual** button at the bottom left of the window, with the app's icon on a book, opens this manual in the computer's PDF viewer.

The window has a fixed size. On a screen too small for it, the window fits the screen and can be scrolled.

### Settings and log files

Retrospective keeps its files in a folder of its own, which it makes at its first start:

| | |
|---|---|
| macOS | `~/Library/Application Support/Retrospective/0.6.1p` |
| Windows | `%APPDATA%\Retrospective\0.6.1p` |

**Settings.** The settings are kept in `retroSettings.json`. The file is created with the standard values at the first start, and can be edited by hand; changes take effect at the next start. Retrospective keeps your choices of the theme, **sounds on/off**, the **message level**, **update notice**, **auto update**, the species preset, **guess reco**, **sigpy reco**, compressed sensing or deep learning, the **NLMF** strength, the temporal **basis**, the LV segmentation **model** and **smooth over cycle**, and the navigator movie's **fast** or **slow**, and the folders last used for loading and exporting, for the next start; a new version takes them over. The file holds, among others, the navigator and reconstruction presets, the folder where the file browser opens (`mrdImportPath`, also the folder auto reco watches), the message level, the number of log files kept, and the deep learning switch (`AImodule`). Five entries apply to the LV segmentation (step 8): `lvUseGpu`, use the graphics card when it can be used (true); `lvMyoDensity`, the myocardial density for the mass (1.05 g/mL); `lvSmoothFrameSelection`, median filter the blood pool volume curve when choosing the ED and ES frames (true); `lvModel`, the segmentation network (`lv_segment_v4c5.onnx`, v4c5, `heart_model`, v4c5 with the heart model, `lv_segment_v3a.onnx`, v3a, or `lv_segment.onnx`, v1; a name that is not one of the networks the app comes with falls back to v1, with a warning); `lvConsistency`, smooth the segmentation over the cardiac cycle before the labels are chosen (true).

Six entries apply to the ejection and filling rates (step 8): `lvFillingHarmonics`, how finely the volume curve follows its frames where the usual reading cannot be made (8); `lvFillingMinFrames`, the frames the diastolic parameters need (32); `lvEASeparation`, how deep the dip between E and A must be, as a fraction of the smaller peak (0.2); `lvNoiseFlagPercent`, the noise per frame, in % of EDV, above which the rates are flagged (1); `lvShiftFlagMs`, the slice timing offset above which it is flagged (10 ms); `lvMidSlice`, the mid-ventricular slice, counted from 1 (0: chosen by the app). A file that cannot be read is set aside as `retroSettings-unreadable-<time>.json` and replaced by the standard values.

**Log files.** Every session writes a log file, `retrospectivepy-log-<date and time>.txt`, to the same folder. The log keeps every message, including the ones the message box does not show at the chosen message level. The 50 most recent log files are kept.

**3D UTE readout profile.** The readout profile for 3D UTE data, `ktraj.txt`, is in the same folder. The first 3D UTE load writes it with the app's own profile. It holds the k-space radius of every readout sample, one sample per line (0.5 is the edge of k-space), in one column for all gradient axes or in three for x, y and z; lines starting with `%` are comments. Its comment line `% sample period 2.5 us` says which sample period the profile is for: a scan with another sample period gets the profile resampled to its own, and the message box says so. A measured profile of your own can replace the file; a comment line `% data offset 6` (decimals allowed) then names the data offset it belongs to, which the app uses for scans at the profile's own sample period. Without such a line the app finds the data offset from the data, which makes the sort take longer. Delete the file to get the app's own profile back. A `ktraj.txt` that holds the profile of an earlier version of the app and names no sample period is replaced by the app's own at the next 3D UTE load, which the message box says.

**Settings to set after a phantom scan.** Three entries of the settings file depend on the scanner and are set once, from a phantom scan, for the radial sequences of "Sequences with an acquisition of their own" (step 1). Until they are set, the app uses the standard values. Edit them with the app closed (the file is read at the start), then load the phantom scan again to check them.

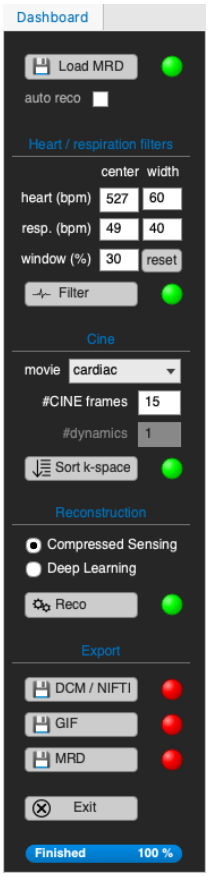
- `radialAngleFlip` (0 or 1) and `radialAngleOffset` (degrees, −360 to 360) give the direction of the golden-angle and center-out spokes against the image axes, for aumc_flash_004 and aumc_flash_005. Set them after the first radial scan of a phantom that is not symmetric (for example a bottle with an object at one side), next to a Cartesian scan of the same slice: when the radial image is mirrored against the Cartesian one, set `radialAngleFlip` to 1; when it is turned, set `radialAngleOffset` to the angle that turns it into place (the image turns with the spokes; when it turns the wrong way, use the opposite sign). With either set, the message box says so at every load.
- `centerOutRampOffsets` gives, per sample period, the time from the start of the readout gradient's ramp to the first sample of a center-out spoke (aumc_flash_005 with the read prephaser off). It is a list of rows of two numbers, the sample period in tenths of a microsecond (75 for 7.5 µs) and the offset in µs, for example `[[75, -4], [50, -3]]`; a sample period not listed has 0. Set a row after the first center-out scan of a phantom at every sample period you use: reconstruct the phantom with a few offsets (for example −8, −4, 0 and 4 µs), and take the one that gives sharp edges, an even intensity and the phantom's true size. The load message gives the ramp offset used.

Two more logs go to the temporary folder: `retropyPythonSetup.log` (the setup of the deep learning environment, section 5) and `retropyLVseg.log` (the last LV segmentation).

## 7. Basic operation

The Retrospective app operates from a single window with 5 main panels.




**Panel 1: Dashboard.** The task buttons that control the reconstruction process. The dashboard tasks need to be completed from top to bottom. A green light next to a task indicates that it has been completed; yellow, that it is busy; red, that it has not been completed yet. Under **Load MRD** is the **auto reco** box (section 8). At the bottom of the Dashboard one progress bar shows the progress of every task: what it is doing and how far it has gotten ("Filtering", "Reconstruction 40 %", "Export 30 of 480", "Finished").

**Panel 2: Navigator / Heart / Respiration / Settings.** The raw navigator signal, the navigator frequency spectrum, and the navigator and app settings.

**Panel 3: Navigator analysis / Metadata.** The navigator signals filtered for cardiac and respiratory motion, and the heart and respiratory rates as a function of time, with the final heart and respiration rates beside their plots. The **Metadata** tab beside it holds the record of the scan: the animal, the session and the contrast agent (step 9).

**Panel 4: Sequence / Geometry / Reco / Radial.** The acquisition and reconstruction parameters.

**Panel 5: Reco / Radial.** The reconstructed CINE movie. The Radial tab shows the k-space trajectory of radial, UTE and ZTE acquisitions.

Below the panels is the message box, newest message on top, with the app's status (OK, WARNING, ERROR, BUSY) beside it.

### Step 1: Loading data

Press **Load MRD** to load the MRD raw data file. The data should include navigator data for cardiac and respiratory synchronization. On the scanner computer (Windows) the first dialog of a session opens in the newest exam of the scanner's MRD folder, unless the settings name a folder (`mrdImportPath`). Without either, it opens in the folder of the scan loaded last.

For data acquired with multiple receivers (phased-array RF coil) select one of the MRD files; the app finds the others.

The relevant acquisition parameters are shown in the Sequence tab of panel 4, including the k-space trajectory: linear, zigzag, user defined, P2ROUD (from the table beside the MRD file, whatever its name), 2D radial, 3D UTE (from a `lut*.txt` file there) or 3D ZTE.

#### Sequences with an acquisition of their own

Scans of the following sequences are recognized from the scan's protocol, and the message box says at the load how the app reads them. A protocol setting the app cannot reconstruct is refused at the load, with an ERROR that names the setting; the scan is then not loaded.

**Slice-interleaved multi-slice CINE (aumc_flash_003).** With **batch slices** 0 (or at least the number of slices), the scan acquires every slice within every TR, one after the other, instead of one slice after the other. The message box gives the number of slices per TR, the time of one slice's slot and the order in which the slots acquire the slice positions, for example "Interleaved multislice: 8 slices per TR, slot 0.75 ms, order 2 4 6 8 1 3 5 7". At message level 2 it also says whether the first repetition is kept (the scan has dummy passes) or left out of the sort (the scan has none, and the slices are still reaching their steady state during it). Every slice shares one navigator signal and therefore the same heart and respiration rates: **filter per slice** (Nav settings tab) is not used, the rate plots show no slice shading, and **Segment LV** gives all slices one common shift, as with an external navigator (section 11.3.2). Both the external navigator and the in-slice (slice FID) navigator can be used. With **batch slices** 1 the scan is an ordinary multi-slice scan. Refused: groups of slices (**batch slices** between 2 and the number of slices minus 1), interleaved slices with the frame loop on or without steady state, and interleaved slices in any other than a 2D multi-slice scan.

**Golden-angle radial CINE (aumc_flash_004).** With radial on, the scan acquires 2D radial spokes, single slice, multi-slice (slice by slice or interleaved, as aumc_flash_003) or as a stack of stars (several partitions, reconstructed as a 3D volume). All spokes are used, those of the first repetition included. **phase steps 1 order** chooses the angles: 1 the golden angle, 2 the tiny golden angle, 0 uniform spokes over a half circle (not in a stack of stars). The message box gives the angle between views, the spokes per repetition and the repetitions, and for a stack of stars its partitions. The scan needs a navigator (slice FID, external FID or external echo). The Radial tab offers the same settings as for other 2D radial scans (see "The Radial tab"); **realtime** is not offered. How the spokes lie against the image axes is set in the settings file (`radialAngleFlip`, `radialAngleOffset`; see "Settings to set after a phantom scan" in section 6). With radial off, the scan is read as aumc_flash_003. Refused: other **phase steps 1 order** values, uniform spokes in a stack of stars, and a scan without a navigator.

**Center-out radial CINE (aumc_flash_005).** With the read prephaser on, the scan is read as aumc_flash_004. With it off, every spoke starts at the center of k-space and runs outward, over the full circle, which gives a shorter echo time; the echo time in the Sequence tab is that of the first sample, the center of k-space. The message box says so at the load, with the readout ramp time and the ramp

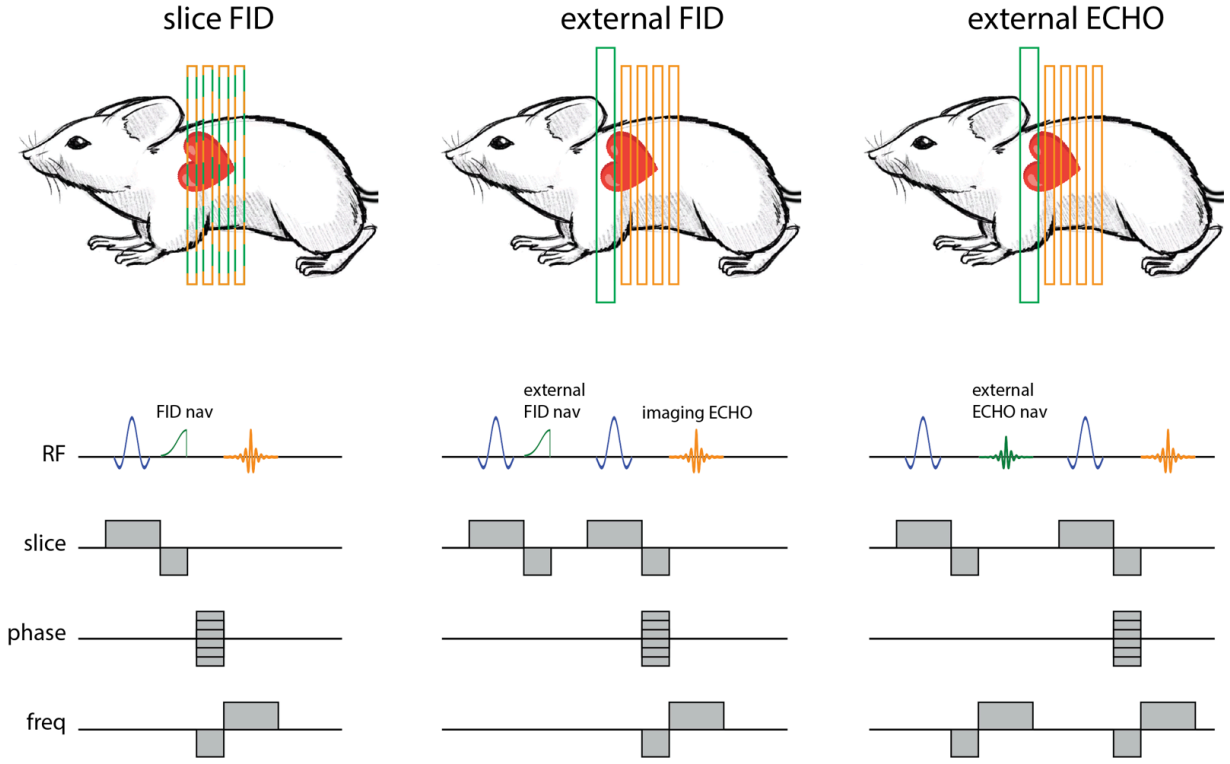
offset taken from the settings file (`centerOutRampOffsets`; see "Settings to set after a phantom scan" in section 6). The spokes' own center samples serve as the self-gating signal: in a scan without a slice FID navigator, the **slice FID** choice in the Navigator tab takes the first samples of every spoke, and an external navigator, when the scan has one, is chosen first. In the Radial tab only **density correction** can be set; the delay fields and **calibration** do not apply, and the message box says so when they are set. **realtime** is not offered. Refused: center-out spokes with uniform angles (**phase steps 1 order** 0).

**3D UTE (ute3d22, ute3d23, aumc_ute3d_001).** The spoke directions come from the `lut*.txt` file beside the scan and the readout along a spoke from the readout profile (`ktraj.txt`, section 6). The app sets the data offset, the readout samples before the profile's first row, at every sort, unless one is typed in the Radial tab (see "The Radial tab"), and at message level 2 the message box says where it comes from. The reconstructed volumes have the orientation of the scanner's own Cartesian scans of the same object, and an off-center 3D UTE scan lies where the scanner placed it.

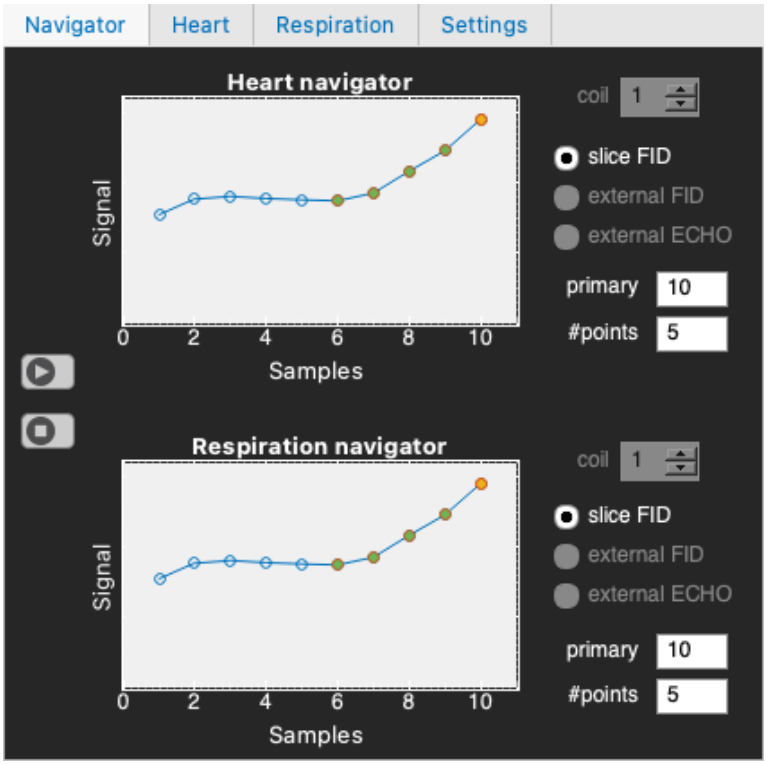
### Step 2: The navigator signal

There are **3** types of navigators, which can be acquired independently and in combination, and in both 2D and 3D imaging.

- **Slice FID navigator** (left): the free-induction-decay (green) generated from the rewinder of the slice selection gradient.
- **External FID navigator** (middle): the free-induction-decay (green) produced by the rewinder of a slice selection gradient in a separate, non-spatially encoded slice.
- **External ECHO navigator** (right): the gradient-echo (green) produced in a separate slice with a readout gradient in a single dimension. The resulting navigator is the Fourier transform of the echo and therefore a 1-dimensional image (readout profile) of the object.



Depending on which type of navigator is available, one can choose slice FID, external FID or external ECHO, separately for heart and respiration, in the Navigator tab. The raw navigator signals and their spectra are shown in the Heart and Respiration tabs; the filtered signals are in panel 3 (step 4).

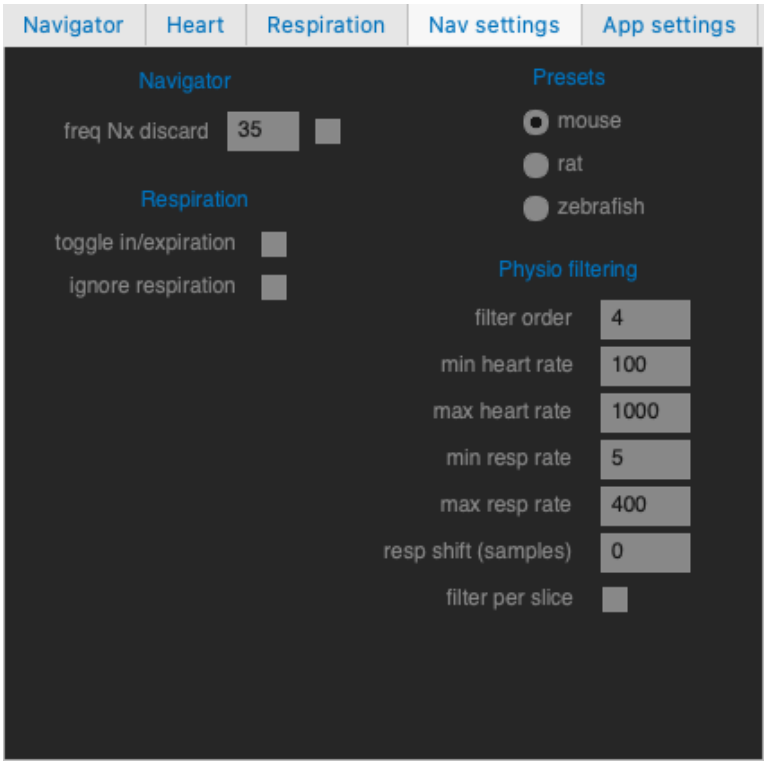



For multi-receiver coil data, individual navigators can be inspected with the **coil** spinner. The app picks the clearest coil or coil combination for heart and respiration.

The respiration peaks in the navigator should be positive. The app turns the navigator the right way up; the **flip** switch (Heart and Respiration tabs) overrules it when needed.

If cardiac and respiratory oscillations are not clearly visible, change the **primary** point slightly and/or increase **#points** (typically 1 to 5) in the Navigator tab, and see whether the quality improves. The play and stop buttons to the left of the plots run through the navigator samples as a movie.

### The Nav settings tab




The settings of the navigator analysis: Navigator and Respiration on the left, Presets and Physio filtering on the right.

**Navigator. freq Nx discard** is the number of samples that are discarded between the end of the navigator and the beginning of the imaging echo; active when checked. If an FID artifact is visible in the reconstructed images, increase this number.

**Respiration.** Images can be reconstructed during in- or expiration. The default is reconstruction during expiration, in the intervals between breaths. For reconstruction during inspiration, check **toggle in/expiration**. Discarding k-lines during respiration can be overruled by checking **ignore respiration**.

**Presets.** Three presets for navigator filtering, for mouse, rat and zebrafish data. They set the min/max heart and respiration rates, which determine the search windows for the heart and respiration rates.

**Physio filtering. filter order** is the order of the Butterworth filters, and **min/max heart rate** and **min/max resp rate** bound the search for the heart and respiration rates; the presets set them. **resp shift (samples)** shifts the respiration window, to correct a delay of the respiration navigator. With **filter per slice** checked, cardiac and respiratory motion are filtered for each slice separately; slice-interleaved scans share one navigator signal and are filtered as a whole (step 1).

### The App settings tab




The window's own settings: Theme, Sounds/messages and LV segmentation on the left, Updates on the right.

**Theme.** Choose a **dark** (the default) or **light** theme for the window.

**Sounds/messages.** Error and warning sounds are switched on or off with **sounds on/off**. The **message level** (1 to 3) sets how much the message box shows: 1 shows the essentials, higher levels more detail. The log file keeps every message, whatever the level (section 6).

**LV segmentation. model** chooses the network that segments the left ventricle (step 8): **v4c5 (slice and frame context)**, the default, which also looks at each image's neighboring slices and frames; **v4c5 + heart model**, v4c5 followed by a model of the shape and motion of the left ventricle, which keeps the base and the apex in the shape of a ventricle and gives a single long-axis slice the model's contours (about half a minute longer per stack, always with smoothing over the cycle); **v3a (short and long axis)**, a network trained on more data, short-axis and long-axis views; or **v1 (original)**, built into the app, which is also used when the network chosen is not found. The message box says which network segmented the images. **smooth over cycle** (on by default) smooths the segmentation over the cardiac cycle before the labels are chosen. Both apply from the next segmentation on, and are kept for the next start (`lvModel`, `lvConsistency` in the settings file).

**Updates.**

- **auto update**, ticked as it comes: new versions of Retrospective arrive by themselves. When one is ready, a window says so with what is new: **Install and Restart** installs it and starts the new Retrospective; **Later** installs it when you quit Retrospective, without asking again. The window waits while the app is working, such as during a reconstruction or an export, or while **auto reco** is on. Without an internet connection nothing is said and nothing waits. On Windows,

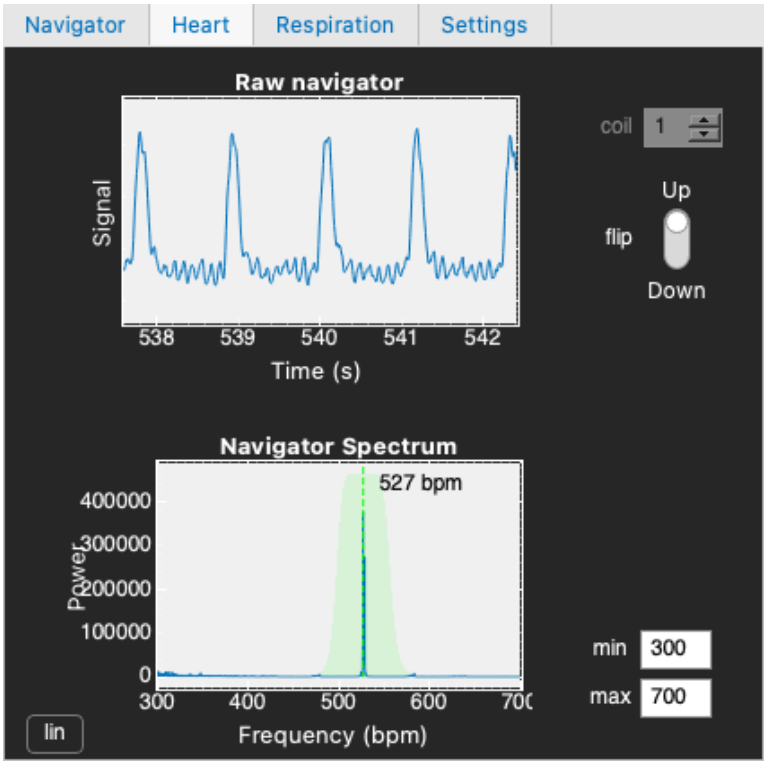
Retrospective installed for all users can be updated only by an administrator; for other users the message box says once where to get the new version to install by hand.

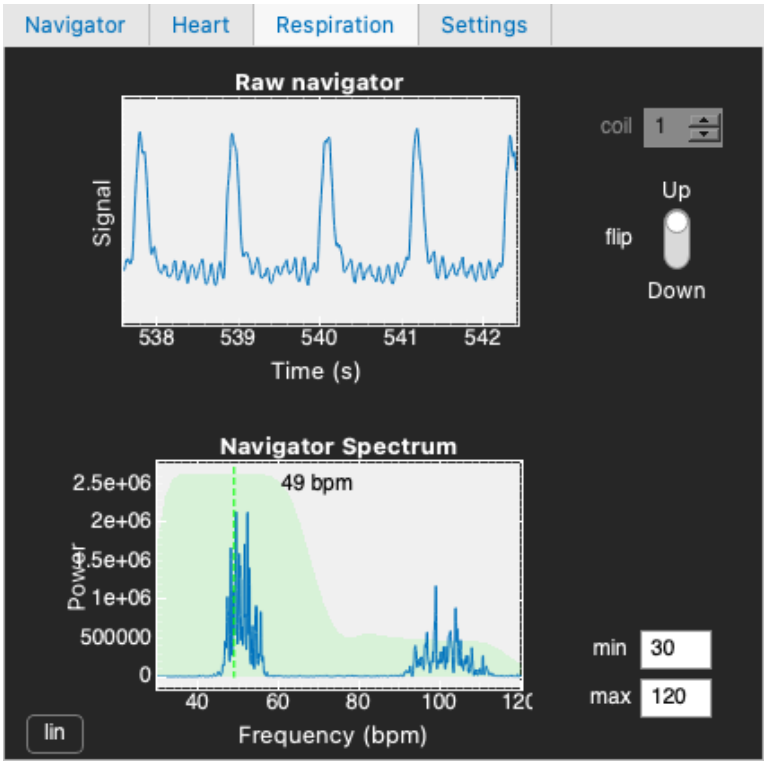
- **Check now** looks for a newer version at once, also with **auto update** off; the message box says what it found. On the Mac, **Check for Updates...** in the Retrospective menu does the same.
- **update notice**, ticked as it comes, counts only with **auto update** off: when a newer version is available, a window at the start offers it. **Download** gets the installer for this computer, **Skip this version** shows no window for that version again, **Later** asks again at the next start.

The message box says at every start whether Retrospective is up to date.

### Step 3: The navigator power spectrum

The Heart and Respiration tabs show the raw navigator signal (top) and its power frequency spectrum (bottom). Typically, the spectrum shows peaks (and higher harmonics) at the respiratory and cardiac frequencies. The app determines the most likely heart and respiration frequencies automatically.



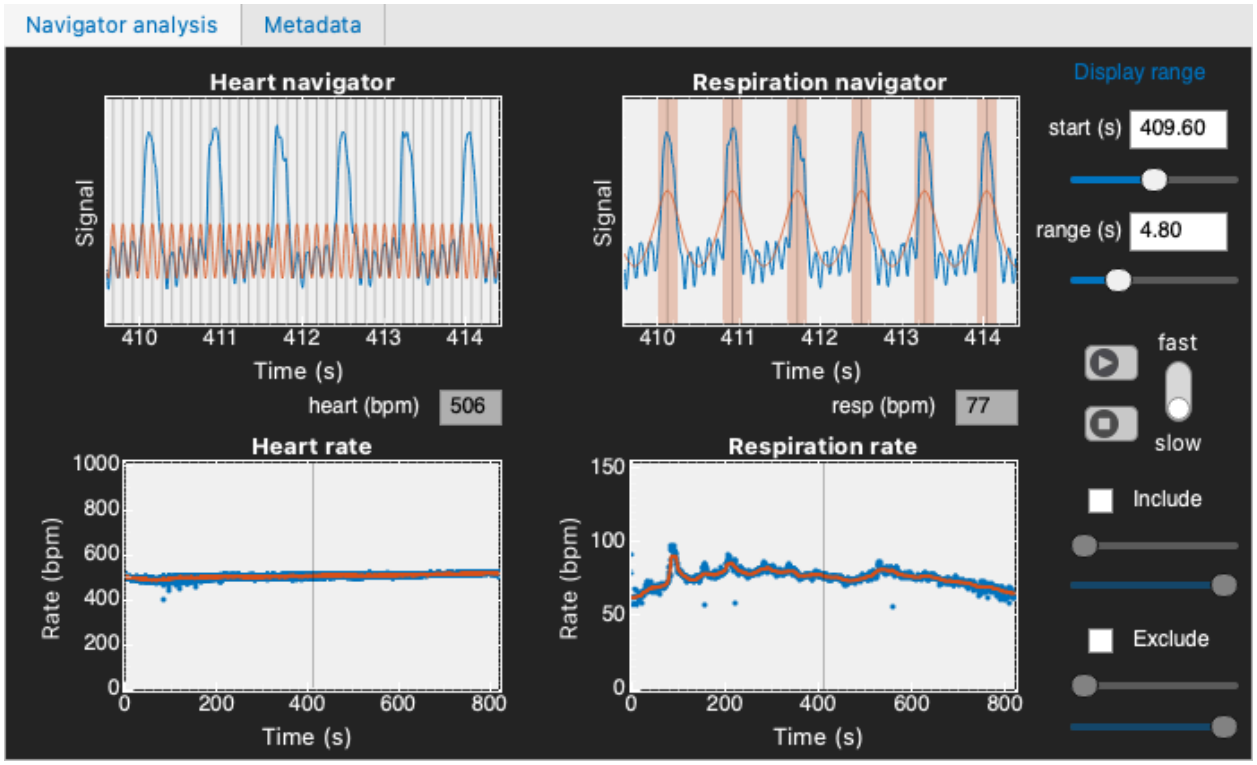



The green shaded area shows the position and width of the Butterworth filter that extracts the periodic cardiac or respiratory motion from the raw navigator. The **lin** / **log** button switches the spectrum between a linear and a logarithmic scale. **min** and **max** set the frequency range shown (bpm); they are for looking only, the search for the rates keeps its own limits (Nav settings tab).

When the automatic respiration and/or cardiac frequency detection fails, the **center** and **width** of the filters can be adjusted in the **Heart / respiration filters** panel of the Dashboard (**heart (bpm)**, **resp. (bpm)**). **window (%)** sets the percentage of the data that is discarded during respiratory motion. Press **reset** to reset the values.

### Step 4: Filtering the navigator

Press **Filter** to filter the raw navigator signal and extract the cardiac and respiratory motion as a function of time. The result may look like this:




The top left plot shows in blue the raw navigator signal and in red the signal filtered for cardiac motion. The gray vertical lines indicate the peaks in the oscillations, which can be used to check the alignment between the red and blue signals. Bottom left is the heart rate as a function of time during the full acquisition: the blue dots are the beat-to-beat values, the red line a smoothed trend line. **heart (bpm)**, above the right end of the heart rate plot, gives the final heart rate of the data used.

The top right plot shows in blue the raw navigator signal and in red the signal filtered for respiratory motion. The shaded red areas indicate the part of the data that is discarded during respiratory motion. Bottom right is the respiration rate as a function of time; blue the breath-to-breath values, red a smoothed trend line. **resp (bpm)**, above the right end of the respiration rate plot, gives the final respiration rate.

Heart and respiration rates should vary smoothly over time. If they do not, adjust the Butterworth filter settings.

The **Display range** sliders and fields (**start (s)**, **range (s)**) change the part of the navigator shown, to inspect the full data set. The play and stop buttons advance the navigator automatically, **fast** or **slow**.

### Including / excluding data

In case of irregular respiration or heartbeat, part of the data can be included (green) and/or excluded (red) by checking **Include** or **Exclude**, or both, and setting the range with the sliders below them. After this, press **Filter** again to filter the data with the new settings. Including or excluding data is not possible for multi-slice 2D data.

### Step 5: Sorting the k-space

Choose the sorting method (**movie**), the number of frames (**#CINE frames**) and the number of dynamics (**#dynamics**) in the Cine panel of the Dashboard. Reconstruction presets are in the Reco tab of panel 4.

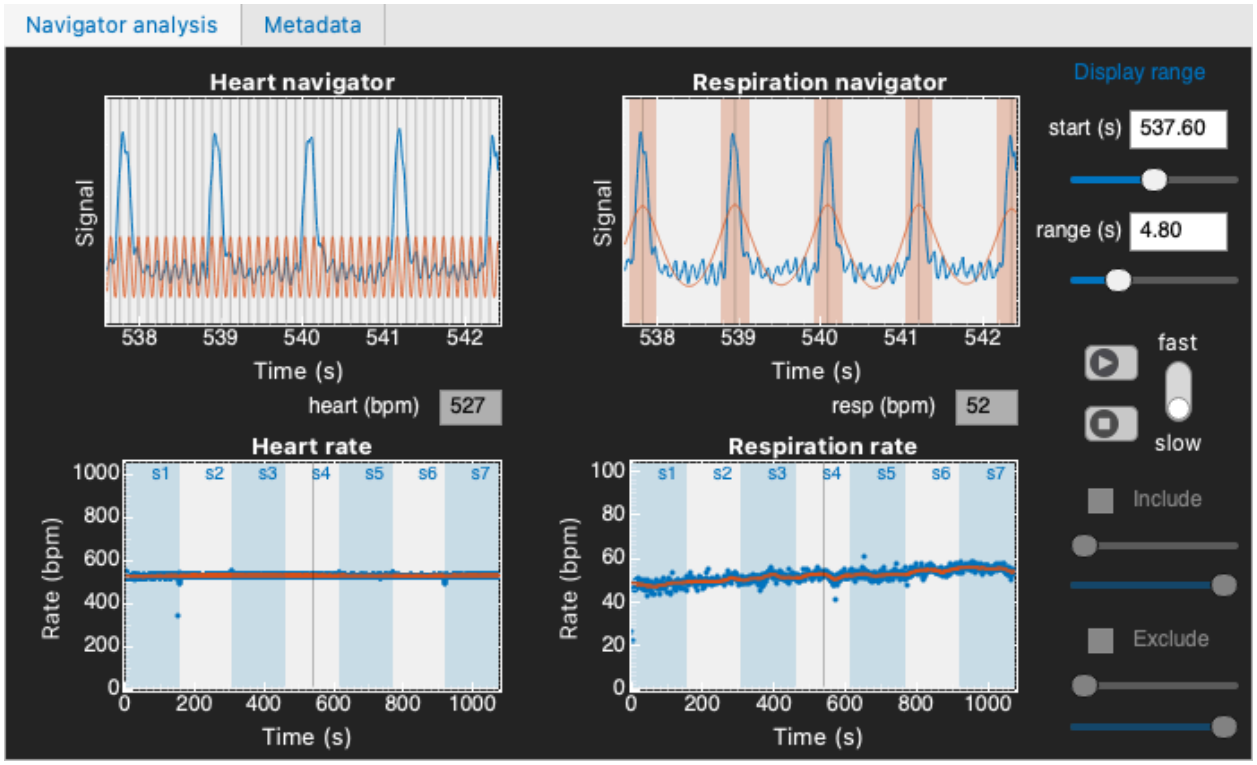
#### Sorting methods

1. **cardiac** — cardiac phase binning. Each heartbeat is divided into an equal number of cardiac CINE frames. The average time between frames follows from the average heart rate and the number of frames. This option is preferred for most applications, particularly for high frame-rate reconstructions for the analysis of diastolic function.
2. **cardiac abs** — cardiac absolute binning. A fixed frame time is set beforehand from the average heart rate and the number of CINE frames. This option can be more accurate for systolic function in the presence of heart-rate variations, but leads to temporal blurring in the diastolic phase.
3. **respiratory** — respiratory phase binning. Each respiration cycle is divided into an equal number of respiratory frames. **window (%)** is then not used.
4. **realtime** — the data is sorted into one CINE movie per heartbeat, for fast dynamic imaging, for example during injection of a contrast agent. This option requires a TR as short as possible and a sparse k-space trajectory that gives sufficient k-lines per frame for compressed sensing and/or parallel imaging reconstruction. The number of dynamics is set to the total number of heartbeats in the acquisition, and respiratory motion is disregarded. **#frames** under Frame sharing, in the Reco tab of panel 4, sets how many neighboring dynamics share k-lines, for better k-space filling and image quality at the expense of temporal averaging. Realtime is available for 2D single-slice scans, Cartesian or radial (not the spokes of aumc_flash_004 and 005); for other scans it cannot be chosen, and when a scan of another kind is loaded after realtime was chosen, the reconstruction preset suited to that scan is selected instead, with a note in the message line. For a radial scan, each image takes the spokes of its own heartbeat and of the number of neighboring heartbeats set under Frame sharing; the realtime preset sets enough of them to cover all spoke angles. More shared heartbeats give fewer streaks but less temporal detail.

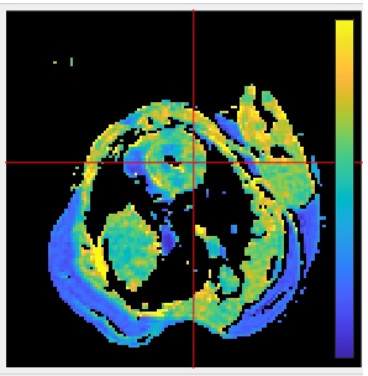
Press **Sort k-space** to sort the data into k-space.

**Dynamics.** Data can be reconstructed as a dynamic time series, for example for dynamic contrast-enhanced acquisitions. Choose the number of dynamics, then sort and reconstruct. There is no maximum number of dynamics, but k-space becomes very sparse with a high number of dynamics, and the reconstruction may take very long. The dynamics are shown in the heart and respiration rate plots as blue/white shaded areas.

**Multi-slice 2D data.** For multi-slice 2D acquisitions, the heart and respiration rate plots show with blue/white shaded areas when the successive slices were acquired (s1, s2, ...). For multi-slice 2D data it is not possible to reconstruct dynamics, nor to include or exclude part of the scan.




**3D data.** Retrospective reconstructs 3D CINE data as well.



**Variable flip-angle data for $T_1$ mapping.** Cardiac quantitative $T_1$ maps can be acquired with the variable flip-angle method (Coolen et al., NMR Biomed 24, 154–162 (2011)). This requires a 3D self-gated $T_1$-weighted (RF and gradient spoiled) acquisition with variable flip angles (typically 5 angles, for example 2°, 5°, 10°, 15°, 20°), acquired in a single acquisition in which the flip angles are evenly distributed over the k-space repetitions. After reconstruction, export the data as a new MRD file (step 10) for $T_1$ mapping with the MAPapp application.

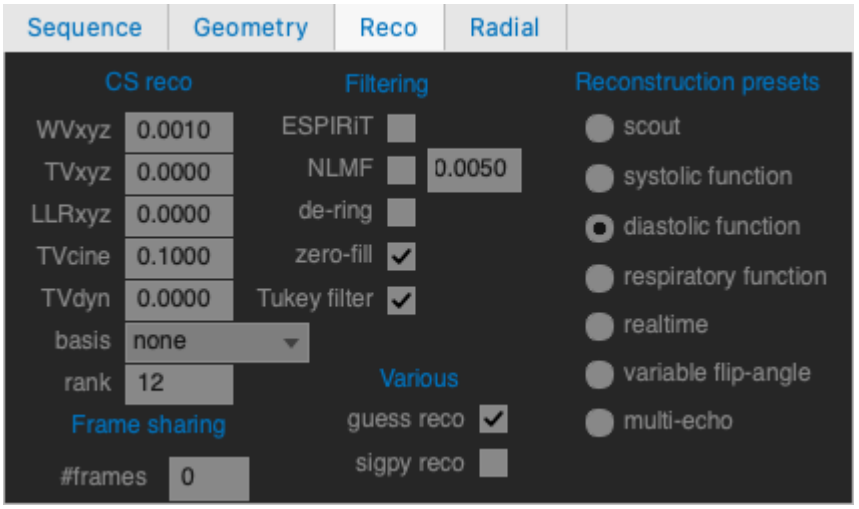
### Step 6: Reconstruction

Before reconstruction, choose the reconstruction settings and regularization parameters in the Reco tab of panel 4. Two reconstruction methods are available, selected in the Reconstruction panel of the Dashboard:

- **Compressed Sensing** — the standard method, described below.
- **Deep Learning** — a trained neural network, described further down.

Press **Reco** to start the reconstruction. While a reconstruction runs, the button reads **Stop**; pressing it stops the reconstruction and leaves the earlier images and the parameters as they were. The button then reads **Stopping…** until the reconstruction has ended; a second press, or a press in the half second after the end, does not start a new reconstruction.

### A. Compressed sensing reconstruction parameters




Every parameter can be used with or without BART (section 5). With the data-derived **basis**, the app picks the engine itself (see **sigpy reco** below).

1. **WVxyz** — wavelet regularization in the spatial (x, y and z) dimensions. For the mouse cardiac CINE, a low (0.001) or zero value gives the best movie quality.
2. **TVxyz** — total variation constraint in the spatial dimensions. Higher values smooth the movie in the spatial domain. For mouse CINEs this does not seem to improve image quality; the recommended value is zero.
3. **LLRxyz** — locally low-rank regularization.
4. **TVcine** — total variation constraint in the CINE dimension. Higher values give better image quality but more temporal smoothing. For mouse CINEs, 0.1 is a good compromise.
5. **TVdyn** — total variation constraint in the dynamics dimension.
6. **ESPIRiT** — ESPIRiT reconstruction (Magn Reson Med 2014; 71(3): 990–1001) of multi-coil data, which includes GRAPPA and SENSE reconstruction.
7. **basis** — the temporal basis of the reconstruction. **none** (the default) reconstructs every frame as set above. **data-derived** describes each pixel over time as a sum of a few curves learned from the scan itself (Han et al., J Cardiovasc Magn Reson 23, 8 (2021)), and reconstructs all frames together. It is meant for sparsely sampled Cartesian series: many frames, or a short scan. All weights apply with a basis, **TVcine** and **TVdyn** included. Recommended: for sparsely filled frames, the basis with **TVcine** 0.1 (as the presets set it), which lowers the noise at the cost of some fine detail of the motion, and **LLRxyz** 0.01 for a quieter background; for well-filled frames and for radial scans, **none** with TVcine.
8. **rank** — the number of curves the data-derived basis uses. After sorting, the app proposes a rank from the scan and puts it in the field; the message box says what it proposes and why. The proposal depends on **TVcine** and **TVdyn**: with them the app proposes more curves (at most 12), as these weights keep the extra curves from adding noise; switching them on or off updates the proposal. The proposal is a starting point: you can type another rank, and the reconstruction uses the number in the field. A number you typed stays until a new proposal is made (you sort again, load a scan, choose the basis again, or switch TVcine or TVdyn on or off); the message box then says that the proposal replaced it.

The **basis** and **rank** controls are available once the k-space is sorted, for 2D and 3D Cartesian and 2D radial scans with at least 4 frames (or dynamics), with compressed sensing. Elsewhere they are grayed out, and their tooltip says why. The basis choice is kept for the next start; the rank is proposed anew for every scan.

### B. Deep learning reconstruction

Select **Deep Learning** in the Reconstruction panel. The deep learning reconstruction uses DRIM, a Dynamic Recurrent Inference Machine trained on rodent cardiac CINE data.

**Requirements.** 2D Cartesian data, single or multi-slice, with at least 4 CINE frames; for 3D, radial, UTE and ZTE data the option is grayed out. It uses the graphics card when it can (section 5), else the processor.

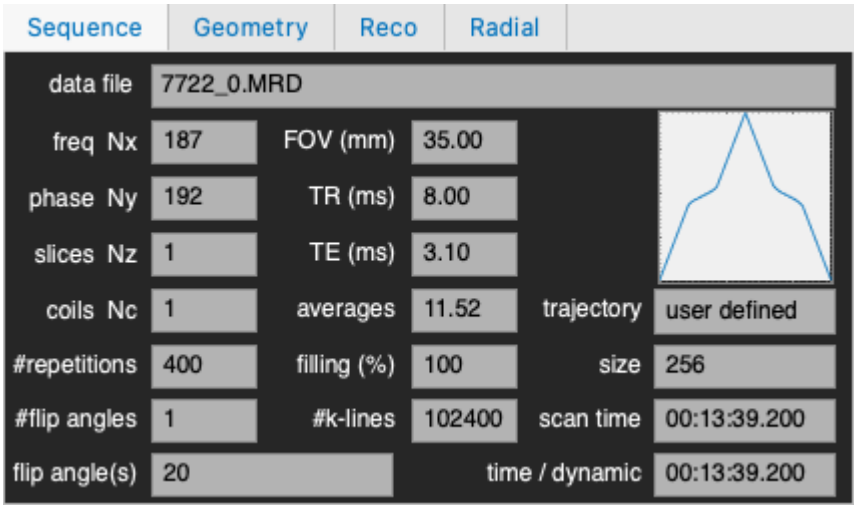
The app chooses the trained model automatically, so there is nothing to set.

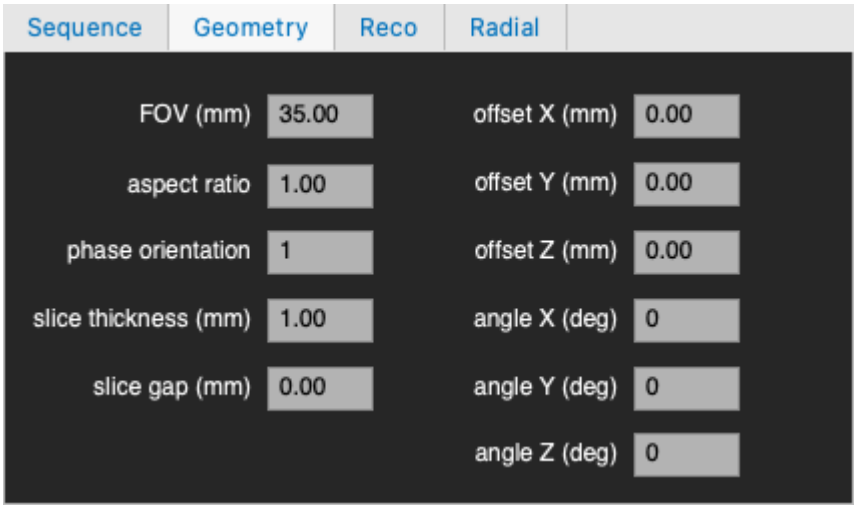
### Points to be aware of

- With **zero-fill** checked in the Reco tab the images come out at 256 x 256, otherwise at the acquired matrix.
- Multi-coil and multi-dynamic data take proportionally longer.
- Memory use grows with the number of CINE frames, and the reconstruction time also with the number of slices. When a reconstruction would not fit in memory, the app does not start it, and the message box says how many frames would fit: fewer frames help, a smaller matrix does not.
- Of the settings in the Reco tab, the compressed sensing weights, ESPIRiT and **sigpy reco** do not apply to the deep learning reconstruction; **NLMF** and **de-ring** are applied after it as well.
- The message box says which device the reconstruction runs on.

### The Sequence and Geometry tabs

The Sequence and Geometry tabs show the sequence parameters and the image geometry.






### The Reco tab

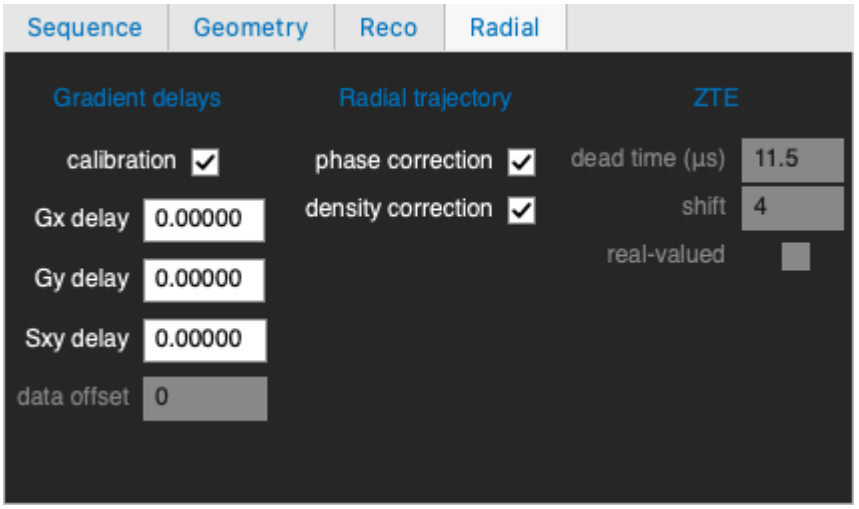
The Reco tab (shown under A above) holds the reconstruction settings besides the compressed sensing weights.

- **basis**, **rank** — the temporal basis of the compressed sensing reconstruction (A above).
- **Frame sharing** — cardiac and/or respiratory motion states can be binned with k-lines from neighboring frames (Küstner et al., NMR Biomed e4409 (2020), doi:10.1002/nbm.4409), to increase k-space filling and SNR. **#frames** sets the number of shared frames.
- **Reconstruction presets** — predetermined reconstruction settings for different applications: scout, systolic function, diastolic function, respiratory function, realtime, variable flip-angle. For a single slice, **diastolic function** sets 64 frames, TV over frames 0.1, no frame sharing and no temporal basis: the settings for E/A. For a stack it sets 32 frames.
- **NLMF** — a non-local means filter on the images, with the strength in the field beside it.
- **de-ring** — Gibbs ringing suppression. Checking it after sorting discards the sort; sort again. With de-ring on, the Tukey filter is not applied.
- **zero-fill** — zero-filling of k-space.
- **Tukey filter** — ellipsoid Tukey filtering of k-space.
- **guess reco** — the app makes an educated guess which type of reconstruction is appropriate; this is especially useful for automatic reconstruction.
- **sigpy reco** — reconstruct with the app's own sigpy engine instead of BART. Without BART it is checked and cannot be changed. The box is a preference: when the settings need the other engine, the app uses that one, and the message box says which engine runs and why. With the data-derived **basis** and **TVcine** or **TVdyn** the reconstruction runs with sigpy, as only sigpy can apply them with a basis; with the basis and neither of them it runs with BART when BART is installed, as BART is faster.

### The Radial tab

The Radial tab holds the settings for radial, UTE and ZTE reconstructions; ZTE scans have their own **dead time (µs)**, **shift** and **real-valued**. **dead time** is the time from the center of k-space to the first sample kept. Too small a value leaves solid objects dark inside with bright edges, too large a value

makes them bulge in the middle. **real-valued** gives a darker background while the object keeps its signal.




Check **calibration** to estimate the gradient delays of 2D radial data at the next reconstruction. The delays are then shown in the delay fields and the box is unchecked, so that a following reconstruction uses them; Load and Sort k-space check it again. For 3D UTE data the delays are entered in the fields.

The third delay field holds Gz for 3D UTE and the Sxy cross term for 2D radial data, and its label changes with the data. The delays, in samples, are reset when a data set is loaded; with calibration off, the delays in the fields are used as they are.

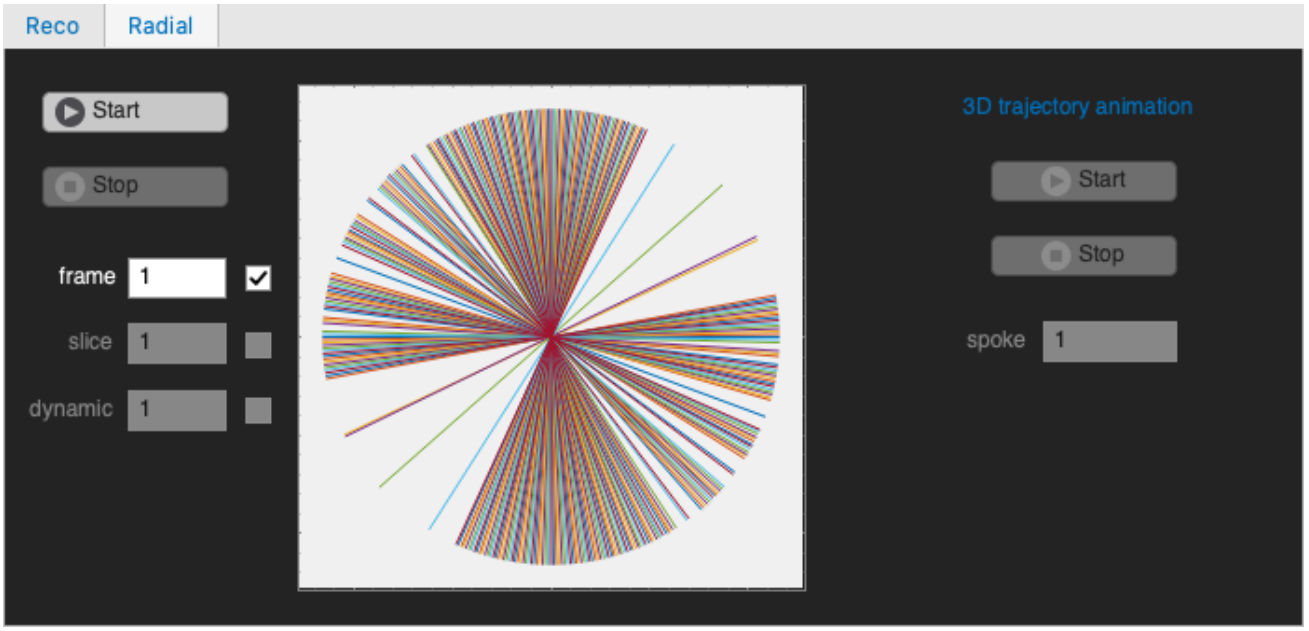
For 3D UTE, **data offset** is the number of readout samples before the readout profile's first row. The app sets it at every sort: the offset the readout profile's file names (section 6), else the one the scan's protocol gives for the app's own profile (from the scan's readout timing, or the standard value for its sequence and sample period), else, for a profile of your own, one found from the data. The field shows the offset used, with decimals where it is a fraction of a sample (for example 0.6), and at message level 2 the message box says where it comes from. An offset typed in the field (decimals allowed) sorts the k-space again and is used by every sort of the scan instead, until the next load, which starts from the scan's own offset again.

For center-out spokes (aumc_flash_005 with the read prephaser off) only **density correction** can be set; the readout ramp offset in the settings file places the samples instead (section 6).

- **phase correction** — corrects the phase of every spoke before the reconstruction.
- **density correction** — a 2D or 3D density correction in k-space for the non-uniform sampling. On by default.

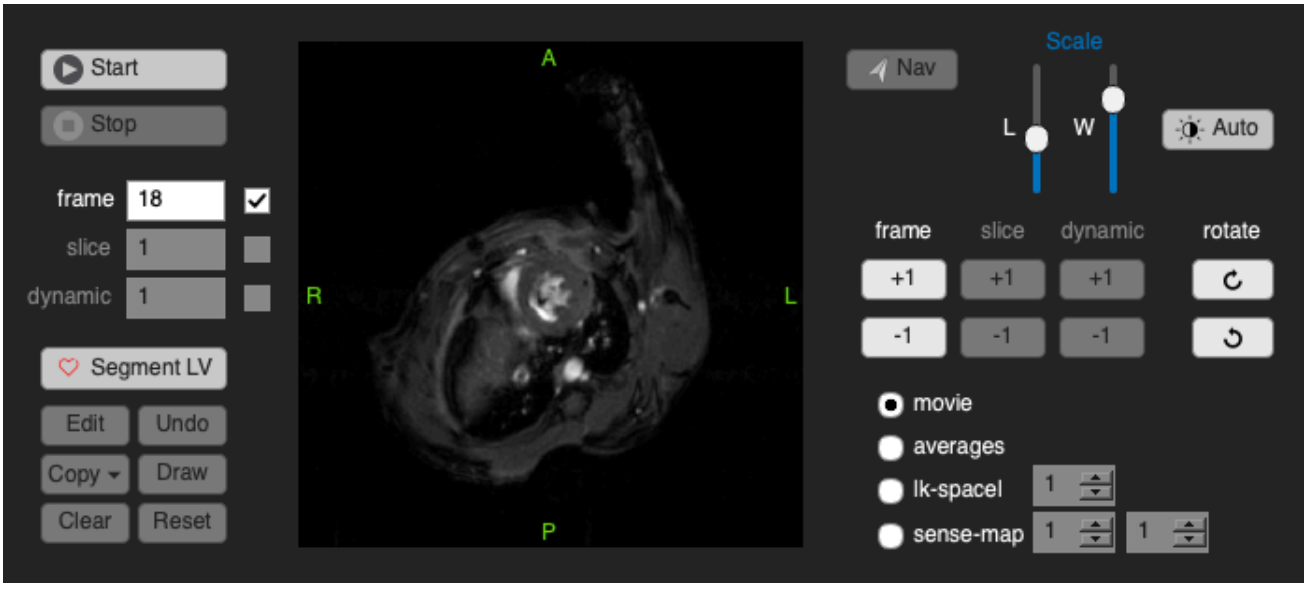
2D radial images are black outside the circle the spokes cover, in the corners of the image.

In the Radial tab of panel 5 the 2D or 3D radial k-space filling can be viewed, and the 3D radial trajectory animated (**3D trajectory animation**); the 3D view turns with the mouse.




### Step 7: Viewing the movie

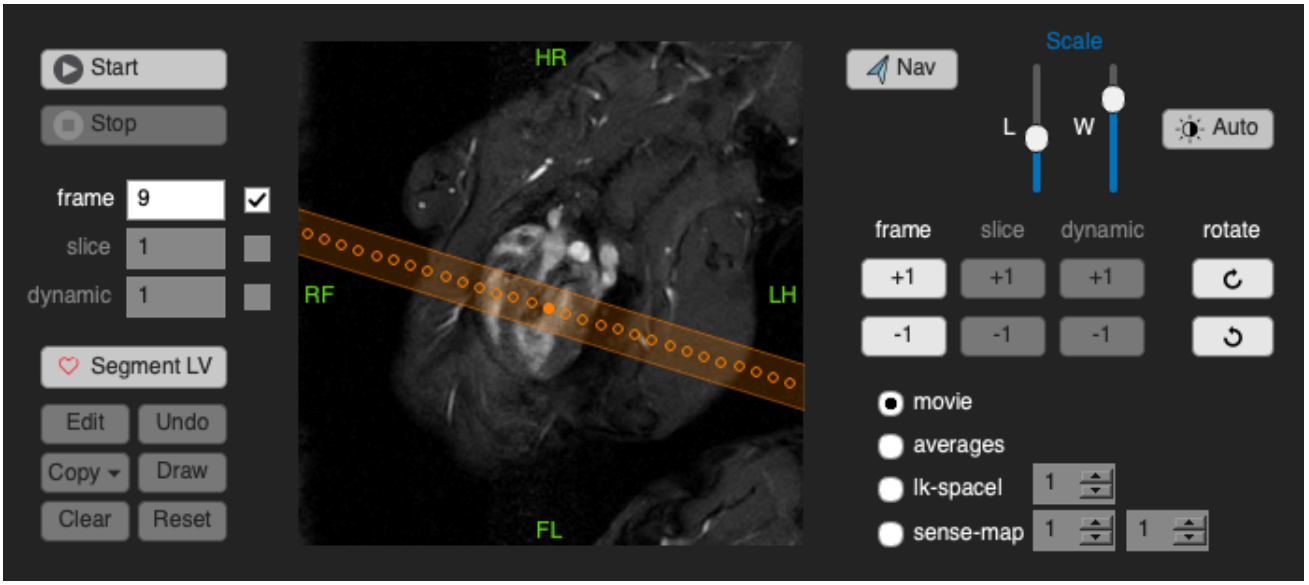
The resulting CINE movie is shown in the Reco tab of panel 5. The radio buttons choose the **movie**, the number of **averages** per k-line and cardiac frame, the **|k-space|**, or the coil sensitivity maps (**sense-map**, ESPIRiT).




- **Start** starts the movie and **Stop** stops it.
- **frame**, **slice** and **dynamic** under them choose the image shown.
- The boxes beside **frame**, **slice** and **dynamic** loop the movie over frames, slices and/or dynamics.
- Image intensity is adjusted with the level (**L**) and window (**W**) sliders; **Auto** sets them from the image.
- The green letters at the edges of the image give the anatomical directions: **L**/**R** left and right, **A**/**P** anterior and posterior, **H**/**F** head and feet. On an oblique plane an edge shows two letters, the

main direction first, when it also points clearly towards a second one. The **o** key shows or hides the letters.

- The rotate buttons turn the movie clockwise or counter-clockwise, its letters with it (this does not affect the DICOM and MRD export orientations).
- **+1** and **–1** below **frame** advance or rewind the movie by one time frame; below **slice** and **dynamic** they step through the slices and dynamics.
- **Nav** shows the navigator slice over the image, when it is in the plane of the current slice.
- **Segment LV** and the editing tools under it (**Edit**, **Undo**, **Copy**, **Draw**, **Clear**, **Reset**) outline the left ventricle and correct the outlines (step 8).
- The scroll wheel, or a box dragged over the image, zooms in; **Whole frame** shows the whole image again. The scroll wheel over a slider moves that slider, one step per notch.




**Synchronizing multi-slice 2D CINEs from one animal**

Since the cardiac synchronization is not based on ECG triggering, CINEs from different slices or acquired in different orientations may not be synchronized with each other: end systole and end diastole may have different frame numbers. **Segment LV** puts this right by itself (step 8): after it, frame 1 is end diastole in every slice. To set the starting phase by hand, first stop the movie with **Stop**. Then set the frame number to 1 in the **frame** field. Then advance or rewind the movie with **+1** or **–1** to the desired starting phase of the cardiac cycle. The new starting frame is used by the DCM / NIFTI, GIF and MRD exports as well. When the slices were already segmented, the volumes and the ejection and filling rates are measured again once you stop shifting (about a second after the last press; while editing, at the next edit or when editing ends), and the message box gives the new values. The exports and **Save LV results** always use the values of the frames as they are.

### Step 8: LV segmentation

Press **Segment LV** to outline the left ventricle in the reconstructed short-axis CINE images and calculate the ventricular volumes. While the segmentation runs, the button reads **Stop**; pressing it stops the segmentation and leaves the earlier outlines, if any, as they were. As with **Reco**, a second press on **Stop** does not start a new segmentation.

The segmentation labels every pixel of every frame and every slice as one of three classes:

| Label | Class |
|---|---|
| 0 | background |
| 1 | left ventricular myocardium |
| 2 | left ventricular blood pool |

**Requirements.** Images must be reconstructed first. Nothing else: the segmentation runs inside the app (section 5). Another network the app comes with can be chosen in the App settings tab (**model**, see "The App settings tab").

Each dynamic is treated as a cardiac cycle of its own and is segmented separately. Any matrix size works, but a field of view larger than 35 mm is cropped to the center: if the heart is not near the center of the image, part of it can be cut off. The message box warns when the field of view exceeds 35 mm.

**Systole first.** After the segmentation the frames are renumbered so that the cycle starts at end diastole: frame 1 is end diastole, followed by systole and then diastole. When each slice was binned on its own slice navigator, the slices are also brought in phase with each other from their systole, each slice moved by its own number of frames. The message box gives the number of frames each slice moved and the timing offsets between the slices (ms). The images themselves do not change, only the order of their frames; a slice whose systole is too weak to time (often the apex or the base) moves with the rest of the stack.

**Slices out of phase and the v4c5 or heart model network.** These networks look at the neighboring slices of an image. When each slice was binned on its own slice navigator, the app first segments with v3a (or v1 when v3a is not there), which looks at one image at a time, brings the slices in phase, and then segments with the network chosen. The message box says each step. This takes about one segmentation longer.

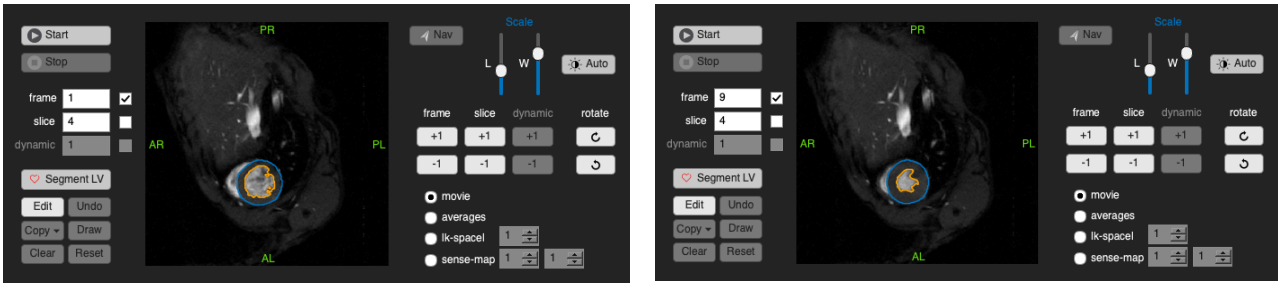
**View check.** The network is made for short-axis images. The message box warns when the images look like a long-axis view, or names the slices that do not look like a short-axis view (for example a slice above the base, or a heart outside the central 35 mm). The segmentation is made all the same; check its contours there.

**Anatomical cleanup.** Only the left ventricle is kept in each slice, and a slice where it was not found (mostly at the base and the apex) is left empty. A slice at either end of the stack that lies beyond the ventricle (above the base, for example) is left empty in every frame. The number of slices that were cleaned or emptied is reported in the message box and written to the results file, so that it can be checked afterward.

### Results

The results of the segmentation are shown over the movie in the Reco tab of panel 5: the endocardium as an orange contour, the epicardium as a blue contour.

**End diastole** **End systole**




*A 7-slice short-axis CINE, slice 4, at end diastole (frame 1) and end systole (frame 9): EDV 77.9 µl, ESV 37.4 µl, EF 52.0 %, ED mass 115.9 mg.*

The following are calculated per dynamic and shown in the message box:

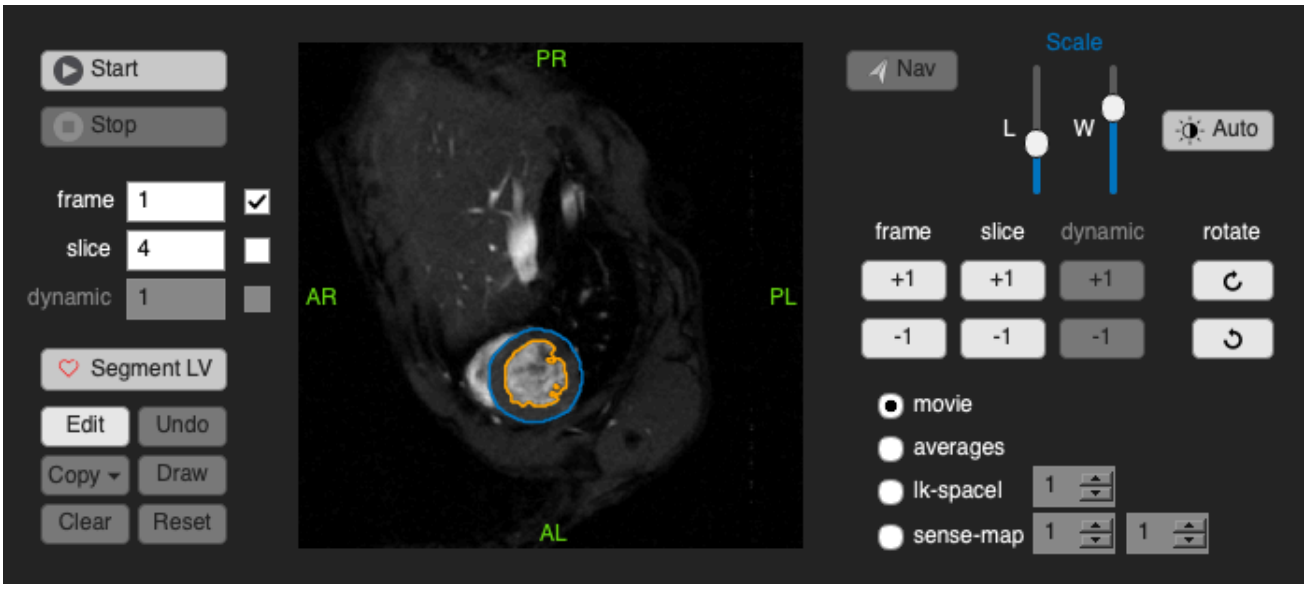
| | |
|---|---|
| EDV | end-diastolic volume (µl) |
| ESV | end-systolic volume (µl) |
| SV | stroke volume (µl) |
| EF | ejection fraction (%) |
| ED mass | left ventricular mass at end diastole (mg) |

After systole first, the ED frame is frame 1 and the ES frame is the frame with the smallest blood pool. After a frame shift by hand (**+1**, **–1**), the ED and ES frames are those with the largest and smallest blood pool. The app warns when a frame chosen as ED or ES has a much larger or smaller blood pool than its neighbors, which can be an over- or under-segmentation rather than end diastole or end systole, and when only one slice was segmented (the values are then single-slice values, not whole-ventricle volumes).

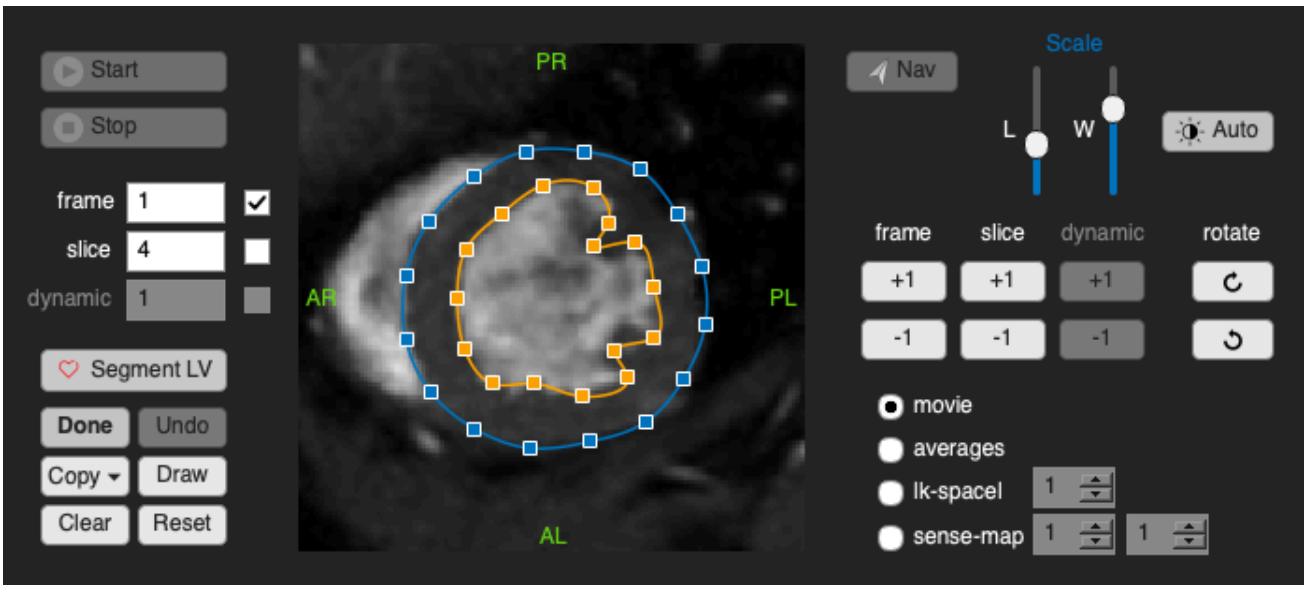
The myocardial mass is calculated with a density of 1.05 g/mL. The settings for the segmentation are listed in section 6.

### Editing the contours

The contours can be corrected by hand after the segmentation, one image (frame, slice and dynamic) at a time: reshaped where they are a little off, removed where the network outlined something that is not the left ventricle, and added where it missed a slice. The editing tools are under **Segment LV**: **Edit** and **Undo**, **Copy** and **Draw**, **Clear** and **Reset**. **Edit** starts the editing and then reads **Done**; press **Done** to end it. **Edit** can be pressed once the images are segmented. The other tools work only while you edit.




1. Press **Edit**. The movie stops and shows the movie view of the image shown; the button stays down and reads **Done**, in bold, while you edit, and the other tools turn on. Clicking another view (averages, |k-space|, sense-map) stops editing and shows that view; clicking **movie** changes nothing. Choose the image to edit with the **frame**, **slice** and **dynamic** fields or their **+1** and **–1** buttons; while editing, **+1** and **–1** below **frame** show the next and the previous frame (after the last frame comes the first) and leave the movie's frames as they are. Each contour now shows small square handles. The movie zooms in on a square around the contours of the slice shown, the same for all its frames; another slice or dynamic zooms in on its own contours, and a slice without contours on those of the slices beside it.




2. **Reshape**: drag a handle with the mouse; the contour follows. When you let go, the contour changes only where you moved it; the rest of it, and the other contour, stay exactly as they were, so dragging the epicardium does not change the blood pool or the volumes. The endocardium is kept inside the epicardium. Dragging anywhere away from the handles zooms in, as outside the editing, and so does the scroll wheel; your zoom stays until you show another slice or dynamic. **Whole frame** shows the whole image again.

3. **Copy** copies the contours of the previous or next slice, or of the previous or next frame, into the image shown (choose from its menu). Then adjust them by dragging.
4. **Draw** draws new contours: click points around the endocardium and close it with a double-click or a click on its first point; then do the same around the epicardium. The contours appear when the epicardium is closed. If you draw the epicardium first, the app notices that the first outline is the larger one, swaps the two and says so in the message box. Press **Draw** again to stop a drawing.
5. **Clear** removes the contours of the image shown.
6. **Undo** undoes the last edit (up to 20 edits) and shows the image it was made on. **Reset** gives the image shown the network's contours back.
7. Press **Done** to stop editing; the button reads **Edit** again. Pressing **Start** also stops it. The view chosen before the editing comes back, with the zoom it had.

After every edit the volumes, the ED and ES frames, EF, the mass and the ejection and filling rates are measured again and shown in the message box, followed by a list of the edited images ("LV segmentation edited by hand: slice 2 (frames 3, 4)"). The exports written before an edit do not have it: their lamps turn red. The frames are not renumbered by an edit.

Pressing **Segment LV** again replaces all edits with a new segmentation; when there are edits the app asks first.

#### Ejection and filling rates

For a cardiac CINE, the message box also gives the rates at which the left ventricle empties and fills, read from the blood pool volume over the cardiac cycle:

| | |
|---|---|
| PER | peak ejection rate: the fastest emptying in systole (µl/s, and in EDV/s) |
| tPER | the time from end diastole to the peak ejection (ms) |
| PFR | peak filling rate: the fastest filling in diastole (µl/s, in EDV/s and in SV/s) |
| tPFR | the time from end systole to the peak filling (ms) |
| E, A | the early and the atrial filling peaks (µl/s) |
| E/A (peaks) | the ratio of the two peaks, E / A |
| AFF | atrial filling fraction: the part of the stroke volume filled by the atrial contraction (%) |
| E/A (volumes) | the volume filled early over the volume filled by the atrium, (1 – AFF) / AFF |

A rate per EDV or per SV can be compared between hearts of different size. EDV, ESV, EF and the mass are those of the contours, as above. From 32 frames on, the rates and E/A do not depend on the number of frames.

**E/A from the peaks and from the volumes.** E/A (peaks) is the classic E/A, the ratio of the peak rates. E/A (volumes) and AFF use the filled volumes, which depend less on the height of a single narrow wave than the peaks do. Report the volumes and AFF alongside the peaks.

**Heart rate.** E/A changes with the heart rate. The message box gives the mean heart rate with the E/A line, and for a scan longer than about two minutes also the mean of its first and its last minute ("506 bpm (first minute 494, last minute 518)"): when they differ, the two halves of the scan have different E/A.

To repeat a measurement on the same scan, compare halves made of alternating blocks of time rather than the first and the second half.

**Full stack and mid slice.** For a stack of slices the rates are given twice: for the **full stack**, the sum of all slices brought in phase, and for **slice N (mid-ventricular)**, the slice with the largest blood pool at end diastole, leaving out the outermost slices at the base and the apex (`lvMidSlice` chooses another). A scan of one slice is labelled **single slice**; its rates per EDV and per SV are the ones to compare with a mid slice.

**At least 32 frames.** The filling parameters (PFR, tPFR, E, A, both E/As, AFF) need at least 32 frames per cycle; with fewer the message box says "Diastolic function: at least 32 frames needed (this scan: N)", and only PER and tPER are given. 48 to 64 frames are recommended. The 32 is a setting (`lvFillingMinFrames`).

**When E/A is not given.**

- "E/A (peaks) not given: E and A fused at ... bpm": the two filling waves merge into one, which happens at the heart rates of mice (above about 550 bpm). AFF and E/A (volumes) are then not given either, since one wave cannot be split into an early and an atrial part.
- "E/A (peaks) not given: A below 15 % of E": the atrial peak is too small to be told from noise. AFF and E/A (volumes) are given, with the flag "A below 15 % of E".

**Flags.** A flag says a value needs care:

- "noise ... % of EDV per frame (above 1 %)": the segmented volume jumps from frame to frame, often a sign of contours that need a look (`lvNoiseFlagPercent`);
- "E sharper than the frames resolve": E rises faster than the frames can follow; E, and so E/A (peaks), are then at least the value given (more frames help);
- "A below 15 % of E", see above;
- "the wave model could not be fitted": the curve could not be read the usual way, and the rates come from a simpler fit; read E/A with care.

**Reliability.** A line per dynamic lists what to keep in mind when reading the rates: the number of frames, the heart rate, the noise per frame, the timing offsets between the slices (marked above 10 ms), the flags, and the TV over frames (tvCine) of the reconstruction. TV over the frames lowers the rates: compare rates only between scans reconstructed with the same settings.

**After an edit.** An image whose contours you edited counts with its edited contours in the curve of the rates.

**How it is calculated.** Section 11 (the appendix) explains in detail how the curve is made, how the wave model is fitted and chosen, and how E, A, E/A and AFF are read from it.

#### Segmentation output files

The segmentation results are written by the **DCM / NIFTI** export (step 10), together with the exported images, so that the numbers and the contours travel with the data.

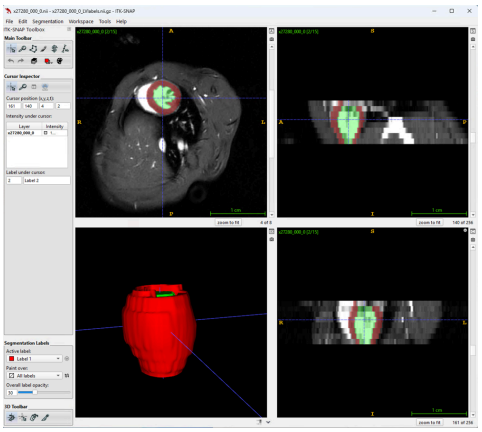
1. **Volumetry** — `LVsegmentation_<tag>.xlsx`, an Excel file with these sheets:

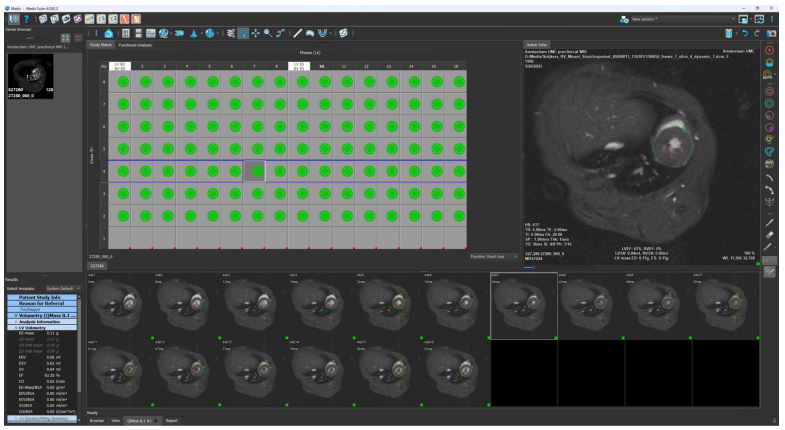
- *Summary*, one row per dynamic, with the columns: DateTime, Tag, Dynamic, EDframe, ESframe, Frames, Slices, EDV_ul, ESV_ul, SV_ul, EF_pct, EDmass_mg, SliceSpacing_mm, ModelPixelSpacing_mm, CleanUp, DeblobbedSlices, ClearedSlices.
- *PerFrame*, the blood pool and myocardial volume of every frame: Dynamic, Frame, BloodPool_ul, Myocardium_ul, IsED, IsES. This is the curve from which the ED and ES frames were chosen.
- *Timing*, per dynamic and slice: the frames the slice moved (FramesMovedEarlier), the part of a frame used for the rates, the slice's timing offset (ms), whether it was brought in phase, and the network of a first segmentation when there was one.
- *Filling*, per dynamic and measurement (single slice, full stack, mid slice): the heart rate (and its first and last minute), the volumes, the rates, E/A (peaks), the early and atrial filling volumes, AFF, E/A (volumes), why a value is not given, the noise, the flags, the reliability line, tvCine and the settings used.
- *FillingFrames*, the volume of every frame as the rates were read from it, the fitted value, the volume of the contours, and every slice.
- *FillingCurve*, the fitted volume curve and its rate of change over the cycle.
- *Edits*, when contours were edited by hand: the Dynamic, Slice and Frame of every edited image.

2. **Contours for Medis qMASS** — `<PatientID>_STD<StudyID>_SER<n>_ACQ1_ITORIGINAL.con`, one file per dynamic, with `_dyn<number>` added to the name when there is more than one. The file contains the endocardial and epicardial contours of every frame and every slice, and can be loaded onto the exported DICOM series.
3. **ROIs for Horos / OsiriX** — `<PatientID>_STD<StudyID>_SER<n>_ACQ1_ITORIGINAL.rois_series`, the same contours in the format Horos uses for its own saved ROIs, written next to the exported DICOM files.
4. **Label maps as NIfTI** — `<name of the CINE file>_LVlabels.nii.gz`, written next to the exported NIfTI CINE, with the same header, so that it can be opened in ITK-SNAP as a segmentation of that image. The label values are the three classes listed above.

The files are written only when the exported images are the segmented ones. The contour files and the label maps include the edits made by hand, and the comment of every exported DICOM image of an edited segmentation says whether its own contours were edited by hand.

**ITK-SNAP** **Medis**





### The LV results folder

The LV results of a scan are also saved in a folder of their own, `LVanalysis/<scan>/`, beside the DICOM, NIfTI and Metadata folders of the **DCM / NIFTI** export: in the folder that export went to (for example `<export folder>/LVanalysis/<scan>/` beside `<export folder>/DICOM/` and `<export folder>/Nifti/`). Before an export of the scan it goes into the exam's data folder (`…/Data/<exam>/LVanalysis/<scan>/`); for data outside the scanner's MRD/Data folders the app asks for the folder, as the DCM / NIFTI export does. The folder is saved:

- with every **DCM / NIFTI** export of the segmented images;
- with **Save LV results** in the menu of the **Segment LV** button (right-click the button), at any time after a segmentation;
- by the command line with `--segment --export <folder>`, under that folder; without `--export` the command line saves no LV results and says so.

A new save of the same scan replaces the files in the folder; files of your own in it stay. The folder name has no spaces: any character other than a letter, a digit, `-`, `_` or `.` in the scan's name becomes `_` (the same holds for the export folders named after the scan).

The folder holds:

| File | |
|---|---|
| `LVsegmentation_<scan>.xlsx` | the workbook described above |
| `LV-summary.png` | every result of the scan on one page (see below) |
| `LV-summary-mid.png` | for a stack: the same page for its mid-ventricular slice |
| `LV-results.txt` | a summary: the scan and its date, the frames, the heart rate (and its first and last minute), EDV, ESV, SV, EF and mass, the rates, E/A (peaks), E/A (volumes), AFF, the candidate curves with their BIC (which is chosen and which read), the flags, the frames moved, the edited images, the reconstruction settings (frames, TV over frames, frame sharing, temporal basis) and the files in the folder |

With several dynamics the summary figures are made per dynamic (`LV-summary_dyn2.png`, ...).

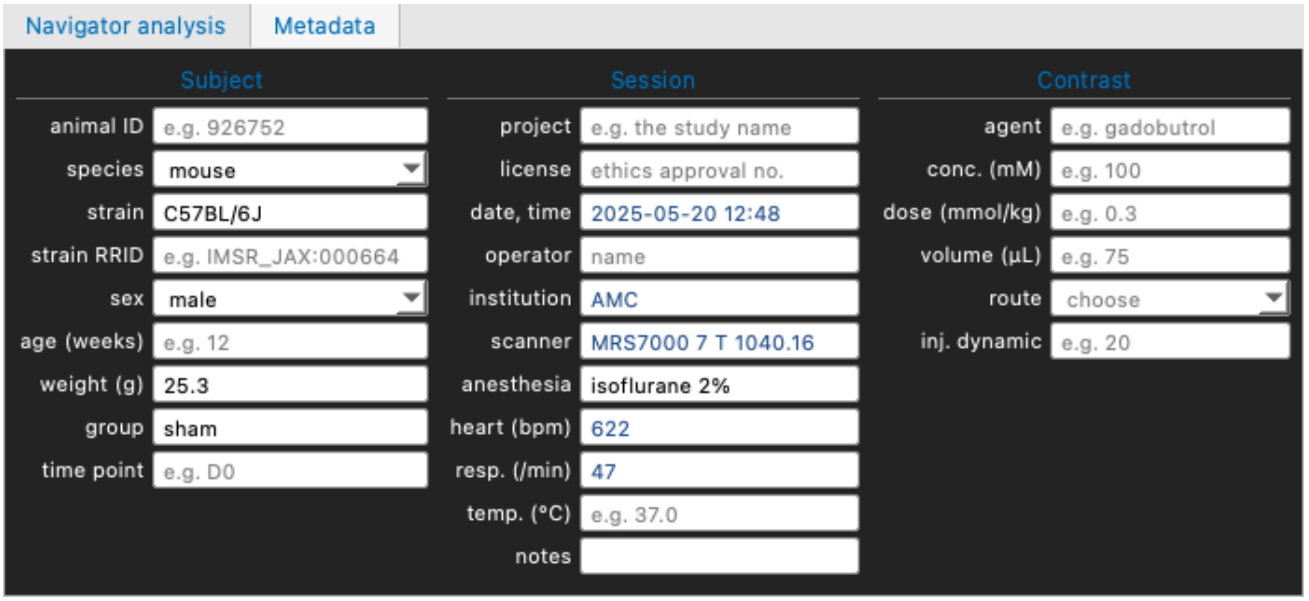
**The summary figure.** `LV-summary.png` is a single landscape page to print or show. Its title gives the scan, whether it is a single slice or the full stack (with its number of slices), the heart rate (with its first and last minute on a long scan), the scan date, the frames and TVcine. On the left are the LV volume over the cycle, with the measured frames as dots, the fitted curve as a line and EDV and ESV written at end diastole and end systole, and the rate of volume change, with PER, E and A marked and the early and atrial filling shaded; thin dashed lines through both panels mark the times of PER, of peak filling and of A, so that they can be found on the volume curve. On the right a table lists every result with its unit: EDV, ESV, SV, EF and mass; PER, tPER, PFR and tPFR, with the rates per EDV and per SV; E, A, both E/As, the atrial filling fraction, EFV and AFV; and the heart rate. A value that is not given is a dash, with the reason in a footnote below the table. A box at the bottom lists what to keep in mind when reading the values, such as a noisy curve; without such points there is no box. For a stack, `LV-summary-mid.png` shows the same for the mid-ventricular slice.

**Reading the summary figure.** Where the measured frames follow the fitted curve, the rates are well supported; frames far from it point to contours that need a look. The shaded areas are the volumes filled early and by the atrium; when slow filling between E and A adds to the early volume, the E area is the E wave's part of it. Section 11.10 lists where each value appears.

### Step 9: Metadata

The **Metadata** tab in the top right panel, beside **Navigator analysis**, holds the record of the scan, which the DCM / NIFTI and GIF exports write with the movie. Its fields stand in three columns:

- **Subject:** animal ID, species, strain and its RRID, sex, age, weight, group and time point;
- **Session:** project, ethics license, date and time, operator, institution, scanner, anesthesia, heart and respiration rate, and temperature, with a field for notes;
- **Contrast:** agent, concentration, dose, volume, route and the dynamic it was injected at.




At a load the app fills in what the scan's files say: the animal ID and the time point (from the patient name typed at the console: `20240212_926752_D0` is animal 926752, day 0), the date and time, the institution, the scanner and the project. What the console leaves at its standard values (sex O, weight 0, the login as the operator) is left empty. After **Filter** the heart and respiration rate are the navigator's mean rates.

What was filled in for an earlier scan of the same exam is filled in again: the animal's details are typed only once per exam. The date, the rates, the temperature and the notes belong to one scan and are never taken from another.

Your institution, operator, anesthesia, species, strain and strain RRID are kept for later exams: they are shown, in blue, from the start, and a scan's own value (from its files or its exam) takes their place where it has one.

A value the app filled in is shown in blue; every field can be corrected, and then counts as yours. Species, sex and route are lists to choose from, or to type in; numbers are kept with their units (a number field holding something else is edged in red). The species and sex are written with their codes (NCBI Taxonomy, PATO), so the record can be read by other software as well as by people.

### Step 10: Export

There are three ways to export the movies, in the Export panel of the Dashboard.

1. **DCM / NIFTI** — exports the data in DICOM and NIfTI format for further processing in third-party software. The app searches for the scanner's DICOM images of the scan. When it finds them, the files go into a new numbered folder beside them, …/Data/<study>/DICOM/<series>R/<n>, and …/Nifti/<tag>R/<n>. When it does not, it asks for a folder. Either way the images lie on the scanner's own pixels, and their position and orientation come from the scan's own parameters. The DICOM files carry the species, strain, sex, age and weight of the animal where the Metadata tab has them. The export also writes the heart and respiration navigators and the cardiac triggers as a BIDS physiological recording, <tag>_physio.tsv.gz with <tag>_physio.json, next to the NIfTI files, and the LV segmentation files and the LV results folder (step 8). After **Segment LV**, frame 1 is end diastole: the DICOM trigger times count from it, and the DICOM image comments say how many frames each slice moved. The MRD export keeps the scan's own trigger times.
2. **GIF** — generates animated GIF movies (one per slice and dynamic for 3D and multi-slice data) of the view on screen (movie, averages or k-space), with the LV contours when segmented. Image intensity (window, level), image rotations and a zoom are preserved. The GIFs are saved in …/Data/<study>/GIF/<tag>R/<n>, or else in a folder you choose.
3. **MRD** — stores the data as a new MRD file in the folder of the input MRD file (or, when that folder cannot be written to, in a folder you choose): retro_<experiments>_<echoes>_<tag>.MRD, with retroData_<experiments>_<echoes>_<tag>_f<filling>.h5, an HDF5 file that holds the k-space, images, averages and parameters. If the app finds an input .rpr file, a new .rpr file is written too. The export also writes the heart and respiration trigger times and rates as text files next to the new MRD file (cardtrigpoints.txt, heartrate.txt, resptrigpoints.txt, resprate.txt, respwindow.txt).

Where an export asks for a folder, the dialog opens on the folder you chose last, also in a later session.

Every export also writes recoparameters_<tag>.txt, the reconstruction settings. The DCM / NIFTI and GIF exports also write, in …/Data/<study>/Metadata/<tag>R/<n> (or under the folder you chose):

- metadata.json — the Metadata tab's record;
- retrospective-report_<tag>.json and .txt — the report of the export: the app and its libraries, the scan and a checksum of each of its files, every step of the session with its main settings, the navigator, filter, cine and reconstruction settings, the heart and respiration rates, and every file the export wrote with its checksum;
- ro-crate-metadata.json — the same as an RO-Crate (https://www.researchobject.org/ro-crate/), a standard description of the folder for data repositories.

The paths in them are relative to the study folder, so they hold no names of your own folders.

### Step 11: Exit

Press **Exit** to close Retrospective. Closing the window does the same, also while the app is busy: the window disappears, and the app closes as soon as the running step has finished. Nothing the app started (BART, the deep learning reconstruction) keeps running after it.

## 8. Automatic reconstruction

When **auto reco** (under Load MRD on the Dashboard) is checked, Retrospective watches a folder and reconstructs every new MRD file automatically, for seamless integration with the MR Solutions preclinical imaging software. It watches the folder `mrdImportPath` of the settings file, or, on the scanner computer (Windows, provisional), the MRD folder of the MR Solutions software; without either, it stops.

Files already in the folder are left alone, and the newest numbered exam folder is followed. Files of Retrospective's or P2ROUD's own exports are ignored. Each new file is loaded, filtered, sorted, reconstructed and exported as MRD. The sounds are switched off, and the window changes color while auto reco runs.

## 9. Sounds and colors

Warnings and errors sound (on a Mac the system sounds Glass and Ping), unless **sounds on/off** in the App settings tab is unchecked.

The window has two themes, **dark** (the default) and **light**, chosen under **Theme** in the App settings tab. It keeps them when the computer is in dark mode.

## 10. Credits

- **BART toolbox and compressed sensing algorithms** — https://mrirecon.github.io/bart/. Michael Lustig, David Donoho, John M. Pauly, Sparse MRI: The application of compressed sensing for rapid MR imaging, Magn Reson Med 58, 1182–1195 (2007).
- **sigpy** — Frank Ong and Michael Lustig, SigPy: a python package for high performance iterative reconstruction, Proc. Intl. Soc. Mag. Reson. Med. 27, 4819 (2019). Used for the compressed sensing reconstruction without BART, and for the non-uniform FFT.
- **Gibbs ringing suppression** — Kellner, E., Dhital, B., Kiselev, V.G. and Reisert, M., Gibbs-ringing artifact removal based on local subvoxel-shifts. Magn Reson Med 76(5), 1574–1581 (2016).
- **Frame sharing** — Küstner et al., NMR Biomed e4409 (2020). doi:10.1002/nbm.4409.
- **Data-derived temporal basis** — Han et al., J Cardiovasc Magn Reson 23, 8 (2021).
- **Segmentation** — Shah W, Stuckey DJ, Yao T, Wrobel M, Deniszczyc E, Feng Z, Anderson S, Campbell R, Muthurangu V, Steeden J. Open-source pre-clinical image segmentation: mouse cardiac magnetic resonance imaging datasets with a deep learning segmentation framework. J Cardiovasc Magn Reson. 2026;28(1):102706. doi:10.1016/j.jocmr.2026.102706.
- **Ejection and filling rates, E/A** (section 11) — Schwarz G. Estimating the dimension of a model. Ann Stat 1978;6:461–464 (the Bayesian information criterion). Coolen BF, et al. High frame rate retrospectively triggered cine MRI for assessment of murine diastolic function. Magn Reson Med 2013;69:648–656. Bacharach SL, et al. J Nucl Med 1979;20:189–193 (frame-rate requirements for peak ejection and filling rates). Schnelle M, et al. J Mol Cell Cardiol 2018;114:20–28 (E/A and its limits at high heart rates in mouse echocardiography).

## 11. Appendix: how E/A is calculated

This appendix explains how Retrospective obtains the early (E) and atrial (A) filling of the left ventricle and their ratio from a retrospectively gated cine MRI scan: from the images to the numbers in the message box, the workbook and the LV results folder.

### 11.1 Overview

E/A is read from the **left-ventricular volume curve over one cardiac cycle**. The rate at which the ventricle fills, dV/dt, has two waves in diastole: early filling (E), driven by relaxation, and atrial filling (A), driven by the atrial contraction. The app finds these waves in five steps:

1. **The volume curve.** The DL segmentation labels the blood pool in every frame. A soft, sub-pixel blood-pool area per frame and slice is summed into the volume curve (section 11.2).
2. **Timing.** The frames are put in cardiac order, starting at end diastole. In a 2D multi-slice stack with an in-plane navigator, each slice is aligned on its own systole first (section 11.3).
3. **The wave model.** dV/dt is described as a sum of smooth flow waves (ejection, E and A) whose shapes, timings and volumes are fitted to the curve. Each frame is modelled as the average over its time bin (section 11.4).
4. **Model choice.** Several candidate models, with and without a separate A wave, are fitted. The Bayesian information criterion picks the one the data support (section 11.5).
5. **Reading.** The peak rates, E, A, their ratio, the filling volumes and the atrial filling fraction are read from the chosen model. Each comes with flags, and a value is withheld when the data cannot support it (sections 11.6 and 11.7).

Two E/A values are reported:

- **E/A (peaks):** the ratio of the peak early and peak atrial filling rates. This is the classical E/A, as with Doppler echocardiography.
- **E/A (volumes):** the ratio of the early and atrial filling volumes. It comes with the atrial filling fraction (AFF), the share of the stroke volume filled by the atrial contraction. These volume measures depend less on the sharpness of single waves than the peaks.

### 11.2 The volume curve

#### 11.2.1 Frames

The scan is binned retrospectively into F frames per cardiac cycle (relative binning): frame f collects the k-lines acquired between the fractions f/F and (f+1)/F of each heartbeat. Each frame is therefore the **average** of the heart over its bin of RR/F, not a snapshot. This matters for narrow waves: in a mouse at 500 bpm (RR = 120 ms), A lasts about 20 ms, while a frame lasts 3.75 ms at 32 frames and 1.9 ms at 64. The model accounts for this averaging (section 11.4.3).

#### 11.2.2 The soft blood-pool area

The DL segmentation (v4c5 by default) gives each pixel a probability of being background, myocardium or blood pool. The labels are the most probable class per pixel. A count of labelled pixels changes only in whole pixels, which in a small mouse slice is a noticeable part of the stroke volume.

For the rates the app therefore uses the **soft blood-pool count**: the sum of the network's blood-pool probability over the pixels of the slice. Three details:

- **Before the time smoothing.** The probabilities are taken before the consistency stage's smoothing over the frames (a Gaussian of one frame). That smoothing is wider in milliseconds at low frame counts and would make E/A depend on the frame count.
- **Inside the LV.** The sum runs only over the final LV region: every frame's labels of the slice, grown by 4 pixels. Outside it the network's probabilities are ignored.
- **Units.** The counts are on the network's grid (0.137 mm pixels) and are converted to µl with the pixel area and the slice spacing, as for the volumes.

EDV, ESV, stroke volume, EF and mass stay on the labels. Only the rates and E/A use the soft curve. An image whose contours were edited by hand uses its edited label count, because the network's probabilities no longer describe it.

#### 11.2.3 Full stack and mid slice

- **Single-slice scan:** the curve is that slice's, labelled "single slice". Its volumes are the slice's; the normalised rates (per EDV, per SV) and the ratios are the values to compare between scans.
- **Multi-slice stack:** two curves are read.
  - **Full stack:** the plain sum of the aligned slices.
  - **Mid slice:** the mid-ventricular slice, by default the slice with the largest end-diastolic blood pool, leaving out the outermost slice with blood pool at base and apex; another slice can be chosen in the settings.

### 11.3 Timing

#### 11.3.1 Systole first

After every segmentation the frames are shifted circularly so that the cycle starts at end diastole: systole (ED to ES) comes first, then diastole. ED is found from a periodic Fourier fit of the volume curve (8 harmonics), at its fractional maximum. The images, labels and k-space move by whole frames; the remaining fraction of a frame is applied to the curve only.

#### 11.3.2 Slice alignment

In a 2D multi-slice stack every slice is acquired separately and binned on its own navigator triggers.

- **Slice-FID navigator.** The navigator comes from each imaging slice itself, so each slice's trigger falls at its own point in the cardiac cycle. In a mouse stack the offsets can be a sizeable part of the cycle (for example ±20 ms of a 120 ms cycle).
  - Each slice is shifted to its own systole: its dV/dt is correlated with the sum of the other slices, weighting only the part from ED to halfway between peak ejection and ES. Diastole is not used for the alignment, so the alignment cannot bias the diastolic measures it feeds.
  - Weak base and apex slices (stroke volume below 15 % of the largest slice's, or a correlation below 0.6) get no shift of their own.
  - When a segmentation model that looks at neighboring slices (v4c5, the heart model) is chosen, the stack is first segmented slice by slice (v3a), aligned, and then segmented with

the chosen model.

- **External navigator** (external FID or echo): one navigator for all slices, so one common shift.

The slices are aligned before they are summed: summing slices that are out of phase smears the waves, lowers the peak rates and can merge E and A.

## 11.4 The wave model

### 11.4.1 Waves

The flow dV/dt over one cycle is a sum of waves. Each wave is a **beta density** stretched over its interval [onset, onset + width]:

$$g(t) = \frac{V}{w}\,\frac{u^{a-1}(1-u)^{b-1}}{B(a,b)}, \qquad u = \frac{t - t_{\text{on}}}{w} \in (0,1),$$

with volume V (the area under the wave), width w, and the shape parameters a = κp and b = κ(1−p). The **skew** p (0.2–0.8) places the peak earlier or later in the interval; the **sharpness** κ sets how peaked the wave is. Its integral, the regularised incomplete beta function, gives the volume curve in closed form, so the model's volume and flow are both exact.

The waves, with time measured from end diastole t0:

| Wave | Volume | Timing |
|---|---|---|
| Ejection | −SV (the stroke volume leaves) | from onset oEj, width wEj, skew pEj |
| Late ejection (models with "+") | a share fL of SV | starting uL·wEj after ejection, width wL: the shoulder many segmented curves show at end systole |
| E | the share fE of the filling | starting ivrt after ejection ends, width wE, skew pE |
| A | the rest of the filling | ending endGap before the next ED, width wA, skew pA |
| D (diastasis, model EDA+) | a share fD | a broad symmetric wave from E's peak to A's peak |
| F (fused models) | all of the filling | one wave from ivrt after ejection to endGap before ED, its own sharpness kF |

The ejection, E and A waves share the sharpness κ = 6; the diastasis wave is broad (κ = 2.5).

The volumes add up to zero: what leaves in ejection comes back in filling, so the volume curve is periodic and returns to EDV at the end of the cycle. The model volume is V(t) = VED + ∫ dV/dt.

### 11.4.2 Parameters and bounds

Times and widths are fractions of the cycle; shares are fractions of the stroke volume. The bounds keep the waves physiological, for example:

- ejection width 0.15–0.6 of the cycle;
- E width 0.04–0.6;

- A width 0.05–0.5;
- isovolumic relaxation 0–0.3;
- the end gap before ED 0–0.08;
- E share 0–1 of the filling;
- diastasis share 0–0.5;
- late-ejection share 0–0.4.

The bounds allow any physiological pattern: a dominant E, a dominant A (impaired relaxation), a tall narrow E with a small A (restrictive filling), a long or short diastasis, or one merged filling wave.

#### 11.4.3 Each frame is the average over its bin

A frame is the mean of the heart over its bin (section 11.2.1), so the model is compared with the frames in the same way: the model's value for frame f is its mean volume over the bin [f/F, (f+1)/F), evaluated on 512 points per cycle (at least 4 per frame). The fitted waves describe the heart itself, not the frames. A sharp wave seen at 32 frames is fitted by the same parameters as at 128, so that E/A does not depend on the frame count.

### 11.5 Fitting and choosing the model

#### 11.5.1 Fitting

Every candidate model is fitted to the frame values by **bounded least squares** (trust-region reflective). The minimised quantity is the residual sum of squares (RSS) between the frames and the model's bin means.

The fit starts from a **fixed grid of starting points**. The timing of the starts comes from the 8-harmonic Fourier fit: ED, ES and the time of the peak filling rate. Different shares and widths of E and A are then tried around it.

- **First round:** every start runs 40 iterations.
- **Second round:** the best three continue to the minimum, at most 300 iterations.
- **Result:** the best fit is kept; of fits with equal residuals, the one from the first start in the grid.

The grid is fixed, so the fit gives the same result every time. Each fit runs to its minimum, so the small rounding differences between computers do not change the result. After a hand edit of one image, the fit restarts from the previous fit's parameters, which takes a fraction of a second.

#### 11.5.2 The candidates

| Model | Filling | Extra |
|---|---|---|
| fused | one filling wave | – |
| EA | E and A | – |
| fused+ | one filling wave | late-ejection wave |
| EA+ | E and A | late-ejection wave |
| EDA+ | E, diastasis and A | late-ejection wave |

#### 11.5.3 Choosing: the Bayesian information criterion

$$\mathrm{BIC} = n\ln\left(\frac{\mathrm{RSS}}{n}\right) + k\ln n$$

Here n is the number of frames and k the number of model parameters (Schwarz 1978). The model with the lowest BIC is chosen. Extra waves lower the RSS but cost k ln n, so a separate A wave is kept only when the data show it.

- **A fused model chosen:** the curve shows one filling wave. E/A is "fused", and the filling-volume measures are withheld (section 11.7).
- **An E/A model chosen (EA, EA+ or EDA+):** E and A are **read** from the better of EA and EA+, the one with the lower BIC.
    - EDA+ takes part in the choice so that a curve with a long, flat diastasis is not called fused.
    - The numbers are not read from EDA+ itself: its diastasis wave takes a share of the filling that depends on how well the frames show the diastasis, and with it the atrial filling fraction would depend on the frame count.

### 11.6 Reading the numbers

The chosen model's volume and flow are evaluated on a fine grid of 2048 points per cycle. All times are in ms, and all rates in µl/s.

#### 11.6.1 End diastole, end systole, ejection and peak filling

- **ED** is the largest volume of the model and **ES** the smallest. Systole runs from ED to ES, diastole from ES back to ED.
- **Peak ejection rate (PER):** the largest emptying rate in systole, −min dV/dt. **tPER** is its time after ED.
- **Peak filling rate (PFR):** the largest filling rate in diastole. **tPFR** is its time after ES.
- **Normalised rates:** PER and PFR per EDV (EDV/s), and PFR per SV (SV/s).

#### 11.6.2 E and A (the peaks)

In diastole the peaks of dV/dt are found with two conditions: a prominence of at least 10 % of the PFR, and a height of at least 15 % of the PFR.

- With two or more such peaks, **E** is the first and **A** the highest after it.
- E and A count as **separate** when the lowest filling rate between them lies at least **20 %** below the smaller of the two peaks.
- Without such a dip the waves have merged and the result is "fused": no peak E/A.

$$E/A_{\mathrm{peaks}} = \frac{E}{A}.$$

The peaks are read from the **summed** model flow, not from the individual waves, so the peak values are what the ventricle shows: where E and A overlap, the E peak includes the start of A and the A peak the tail of E.

### 11.6.3 The filling volumes

The split of the filling between early and atrial comes from the waves themselves; the volumes are reported as shares of the measured stroke volume SV (EDV − ESV from the contours), so that EFV + AFV = SV:

- **AFF** (atrial filling fraction): the volume of the A wave as a fraction of the stroke volume of the fitted curve,

$$\mathrm{AFF} = \frac{V_{\mathrm{A}}}{\mathrm{SV}_{\mathrm{fit}}}.$$

- **AFV** (atrial filling volume): the atrial share of the measured stroke volume,

$$\mathrm{AFV} = \mathrm{AFF} \times \mathrm{SV}.$$

- **EFV** (early filling volume): the rest of the measured stroke volume, the diastasis included,

$$\mathrm{EFV} = \mathrm{SV} - \mathrm{AFV}.$$

- **E/A from the volumes:**

$$E/A_{\mathrm{volumes}} = \frac{\mathrm{EFV}}{\mathrm{AFV}} = \frac{1 - \mathrm{AFF}}{\mathrm{AFF}}.$$

Because the diastasis counts with the early filling, AFF with a long diastasis differs somewhat from a definition that splits the diastasis between E and A.

## 11.7 When a value is withheld or flagged

| Condition | Effect |
|---|---|
| Fewer than 32 frames (lvFillingMinFrames) | PFR, tPFR, E, A, both E/As and AFF are withheld: "diastolic function: at least 32 frames needed (this scan: N)". PER and tPER are still given (from the Fourier fit). |
| E and A fused (a fused model chosen, or no 20 % dip) | Peak E/A, AFF and the volume E/A are withheld ("fused"): a single filling wave cannot be split into E and A. |
| A below 15 % of E (the A wave's own peak) | Peak E/A is withheld: the A peak cannot be told from noise. AFF and the volume E/A are given, flagged. |
| E sharper than the frames resolve (E at its narrowest allowed width) | Flagged: the E peak, and with it the peak E/A, is a lower bound. Prefer the volume measures. |
| Noise above 1 % of EDV per frame | Flagged ("noisy"). The noise is the white-noise level the curve's harmonics from F/4 upward carry, where the heart has little energy. |
| The wave model cannot be fitted | Fallback: the curve is read from its Fourier fit (8 harmonics; one per four frames on short scans; each harmonic divided by sinc(k/F) to undo the frame averaging), with E and A from its peaks; AFF is then the share of the fit's stroke volume filled after the dip between E and A. The result says so. |

The **heart rate** is shown beside E/A, and over a long scan also its **drift**: the mean of the first minute against that of the last. E/A depends strongly on the heart rate: when the first and the last minute differ, the parts of the scan have different E/A, and the value given is their mean. For a repeat measurement on one scan, compare halves made of alternating blocks of time.

### 11.8 Reconstruction settings

The **"diastolic function" preset for single-slice scans** uses 64 frames, TVcine 0.1, no frame sharing and no data-derived temporal basis. These are the recommended settings for E/A:

- **Frames:** at least 32 are required, and 48–64 are recommended; more than 64 are not needed.
- **TVcine 0.1.** Without TVcine (0), single frames can fail to segment at high frame counts.
- **No frame sharing:** sharing blurs the narrow A wave.
- **No data-derived basis:** the basis can merge E and A.

Compare rates and E/A only between scans reconstructed with the same settings. They are recorded with the results.

### 11.9 Limits

- **A step-like E.** When E rises almost as a step, the peak E is set by the segmentation and the frame width, and the peak E/A can read much higher than the volume E/A. The flag "E sharper than the frames resolve" marks such scans; use AFF and E/A (volumes) there.
- **Poor segmentation.** It shows as noise above 1 % of EDV per frame and is flagged. Check the contours of the frames that lie far from the fitted curve in LV-summary.png (the frames and the model's values are in the FillingFrames sheet of the LV workbook).
- **Multi-slice stacks.** The diastolic values of a stack depend on how well the slices are brought in phase. Check the timing offsets and the slice curves in the LV workbook (Timing and FillingFrames sheets), and compare the full stack with the mid slice.
- **High heart rates.** Above about 550 bpm, E and A merge in mice whatever the frame count. Report the heart rate with every E/A.

### 11.10 Where the numbers appear

- **The message box** after Segment LV: the rates, both E/As, AFF, the flags and the heart rate with its drift.
- **The LV workbook** (LVsegmentation[_tag].xlsx):
  - the Filling sheet holds every value, the chosen model, its BIC and the curve's source;
  - FillingFrames holds the frames with the soft and label curves, the model and, for a stack, every slice's curve;
  - FillingCurve holds the model curve and dV/dt on the fine grid.
- **The LV results folder** `<export folder>/LVanalysis/<scan>[_<tag>]/`:
  - the workbook;

- LV-summary.png: every result on one page, with the volume curve, its rate, a table of the values with their units and notes on the reading;
- for stacks, LV-summary-mid.png, the same page for the mid slice;
- LV-results.txt, which also lists every candidate's BIC and residual sum of squares and which model is chosen and which read.

It is written with the DICOM export, from the command line, and with "Save LV results" in the Segment LV button's right-click menu.